

Vidya Academy of Science & Technology

(A unit of Vidya International Charitable Trust)

Approved by AICTE & Affiliated to APJ Abdul Kalam Technological University

Accredited by NAAC with "B++" grade



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

PCCSL307 DATA STRUCTURES LAB

LAB MANUAL

PREPARED BY:

Ms.REMYA K R

APPROVED BY:

HOD, CSE

VISION OF THE INSTITUTION

Progress through education

MISSION OF THE INSTITUTION

To seek, strive for and scale greater heights of quality education

- M1: Seek to foster in every student a profound and practical understanding of knowledge, encouraging critical inquiry and intellectual curiosity.
- M2: Strive to ensure every student develops holistically, cultivating the skills, ethical character, and resilience necessary to thrive in diverse and challenging future environments.
- M3: Scale the commitment to service by inspiring students to apply their knowledge and skills to actively address community needs, promoting ethical leadership and social responsibility.

VISION OF THE DEPARTMENT

Progress through quality education and become a centre of excellence in computing knowledge.

MISSION OF THE DEPARTMENT

- M1: To seek and strive for advancements in technological domains.
- M2: To create socially responsible engineers in the field of computer science.
- M3: To bridge the gap between science and mankind through meaningful innovation and service.

PROGRAM EDUCATIONAL OBJECTIVES (PEOs)

- PEO-1:** Graduates shall practice Computer Science and Engineering in IT industry, public sector undertaking, as an entrepreneur or as a researcher pursuing a successful professional career.
- PEO-2:** Graduates shall demonstrate creativity, develop innovative ideas and work in teams to accomplish a common goal.
- PEO-3:** Graduates shall demonstrate project management skills, lifelong learning and social awareness upholding ethical values.

PROGRAM SPECIFIC OUTCOMES (PSOs)

PSO-1: The ability to apply the knowledge of computational structures and algorithms for solving complex engineering problems in multidisciplinary areas.

PSO-2: The ability to design, develop and evaluate computer based systems using advanced tools and technologies to meet the real world challenges.

PROGRAM OUTCOMES (POs)

Computer Science & Engineering graduates will be able to:

PO1: Engineering Knowledge: Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop the solution of complex engineering problems.

PO2: Problem Analysis: Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development. (WK1 to WK4)

PO3: Design/Development of Solutions: Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required. (WK5)

PO4: Conduct Investigations of Complex Problems: Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions. (WK8).

PO5: Engineering Tool Usage: Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems. (WK2 and WK6)

PO6: The Engineer and The World: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment. (WK1, WK5, and WK7).

PO7: Ethics: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws. (WK9)

PO8: Individual and Collaborative Team work: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.

PO9: Communication: Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences

PO10: Project Management and Finance: Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.

PO11: Life-Long Learning: Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change. (WK8)

DATA STRUCTURES LAB

(Common to CS/CA/CM/CD/CR/AI/AM/AD/CB/CN/CC/CU/CI/CG)

Course Code	PCCSL307	CIE Marks	50
Teaching Hours/Week (L: T:P: R)	0:0:3:0	ESE Marks	50
Credits	2	Exam Hours	2 Hrs. 30 Min.
Prerequisites (if any)	GYEST204	Course Type	Lab

Course Objectives:

To give practical experience for learners on implementing different linear and non linear data structures, and algorithms for searching and sorting.

Expt. No.	Experiments
1	Find the sum of two sparse polynomials using arrays
2	Find the transpose of a sparse matrix and sum of two sparse matrices.
3	Convert infix expression to postfix (or prefix) and then evaluate using stack,
4	Implement Queue, DEQUEUE, and Circular Queue using arrays.
5	Implement backward and forward navigation of visited web pages in a web browser (i.e. Back and forward buttons) using doubly linked list operations.
6	Implement addition and multiplication of polynomials using singly linked lists.
7	Create a binary tree for a given simple arithmetic expression and find the prefix / postfix equivalent.
8	Implement a dictionary of word-meaning pairs using binary search trees.
9	Find the shortest distance of every cell from a landmine inside a maze.
10	We have three containers whose sizes are 10 litres, 7 litres, and 4 litres, respectively. The 7-litre and 4-litre containers start out full of water, but the 10-litre container is initially empty. We are allowed one type of operation: pouring the contents of one container into another, stopping only when the source container is empty or the destination container is full. We want to know if there is a sequence of pourings that leaves exactly 2 litres in the 7 or 4-litre container. Model this as a graph problem and solve.
11	Implement the find and replace feature in a text editor.

12	Given an array of sorted items, implement an efficient algorithm to search for specific item in the array.
13	Implement Bubble sort, Insertion Sort, Radix sort, Quick Sort, and Merge Sort and compare the number of steps involved.
14	The General post office wishes to give preferential treatment to its customers. They have identified the customer categories as Defence personnel, Differently abled, Senior citizen, Ordinary. The customers are to be given preference in the decreasing order - Differently abled, Senior citizen, Defence personnel, Normal person. Generate the possible sequence of completion.
15	Implement a spell checker using a hash table to store a dictionary of words for fast lookup. Implement functions to check if a word is valid and to suggest corrections for misspelled words.
16	Simulation of a basic memory allocator and garbage collector using doubly linked list
17	The CSE dept is organizing a tech fest with so many exciting events. By participating in an event, you can claim for activity points as stipulated by KTU. Each event i gives you $A[i]$ activity points where A is an array. If you are not allowed to participate in more than k events, what's the max number of points that you can earn?
18	Merge K sorted lists into a single sorted list using a heap. Use a min-heap to keep track of the smallest element from each list. Repeatedly extract the smallest element and insert the next element from the corresponding list into the heap until all lists are merged.

Course Outcomes (COs)

At the end of the course students should be able to:

Course Outcome		Bloom's Knowledge Level (KL)
CO1	Model a real world problem using suitable data structure and implement the solution.	K3
CO2	Compare efficiency of different data structures in terms of time and space complexity.	K4
CO3	Evaluate the time complexities of various searching and sorting algorithms.	K5
CO4	Differentiate static and dynamic data structures in terms of their advantages and application.	K3

Note: K1- Remember, K2- Understand, K3- Apply, K4- Analyse, K5- Evaluate, K6- Create

CO- PO Mapping (Mapping of Course Outcomes with Program Outcomes)

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PSO1	PSO2
CO1	3	3	3	3			3				3	3	3
CO2	3	3	3	3			3				3	3	3
CO3	3	3	3	3			3				3	3	3
CO4	3	3	3	3			3				3	3	3

1: Slight (Low), 2: Moderate (Medium), 3: Substantial (High), -: No Correlation

CONTENTS

Expt. No.	Experiments	Page No
1	Find the sum of two sparse polynomials using arrays	10
2	Find the transpose of a sparse matrix and sum of two sparse matrices.	14
3	Convert infix expression to postfix (or prefix) and then evaluate using stack,	21
4	Implement Queue, DEQUEUE, and Circular Queue using arrays.	27
5	Implement backward and forward navigation of visited web pages in a web browser (i.e.back and forward buttons) using doubly linked list operations.	45
6	Implement addition and multiplication of polynomials using singly linked lists.	51
7	Create a binary tree for a given simple arithmetic expression and find the prefix / postfix equivalent.	58
8	Implement a dictionary of word-meaning pairs using binary search trees.	64
9	Find the shortest distance of every cell from a landmine inside a maze.	70
10	We have three containers whose sizes are 10 litres, 7 litres, and 4 litres, respectively. The 7-litre and 4-litre containers start out full of water, but the 10-litre container is initially empty. We are allowed one type of operation: pouring the contents of one container into another, stopping only when the source container is empty or the destination container is full. We want to know if there is a sequence of pourings that leaves exactly 2 litres in the 7 or 4-litre container. Model this as a graph problem and solve.	75
11	Implement the find and replace feature in a text editor.	81
12	Given an array of sorted items, implement an efficient algorithm to search for specific item in the array.	85
13	Implement Bubble sort, Insertion Sort, Radix sort, Quick Sort, and Merge Sort and compare the number of steps involved.	88
14	The General post office wishes to give preferential treatment to its customers. They have	97

	identified the customer categories as Defence personnel, Differently abled, Senior citizen, Ordinary. The customers are to be given preference in the decreasing order - Differently abled, Senior citizen, Defence personnel, and Normal person. Generate the possible sequence of completion.	
15	Implement a spell checker using a hash table to store a dictionary of words for fast lookup. Implement functions to check if a word is valid and to suggest corrections for misspelled words.	101
16	Simulation of a basic memory allocator and garbage collector using doubly linked list	108
17	The CSE dept is organizing a tech fest with so many exciting events. By participating in an event, you can claim for activity points as stipulated by KTU. Each event i gives you $A[i]$ activity points where A is an array. If you are not allowed to participate in more than k events, what's the max number of points that you can earn?	115
18	Merge K sorted lists into a single sorted list using a heap. Use a min-heap to keep track of the smallest element from each list. Repeatedly extract the smallest element and insert the next element from the corresponding list into the heap until all lists are merged.	117

EXP NO:1**POLYNOMIAL ADDITION USING ARRAY**

AIM: To Find the sum of two sparse polynomials using arrays

LEARNING OUTCOMES

Students will be able to represent polynomials using arrays and implement an efficient algorithm or program to add two polynomials by combining their corresponding coefficients correctly.

ALGORITHM

1. Initialize indices: $i = 0$ for polynomial A, $j = 0$ for polynomial B.
2. Compare exponents:
 - If $A[i].exp > B[j].exp$, add $A[i]$ to result, increment i .
 - If $A[i].exp < B[j].exp$, add $B[j]$ to result, increment j .
 - If $A[i].exp == B[j].exp$, add coefficients $A[i].coeff + B[j].coeff$:
 - If $sum \neq 0$, add to the result.
 - Increment both i and j .
3. Copy remaining terms: If one polynomial has leftover terms, copy them directly to the result.
4. Print or return the result array.

PROGRAM

```
#include <stdio.h>

typedef struct {
    int coeff;
    int exp;
} Term;

int main() {
    Term poly1[20], poly2[20], result[40];
    int n1, n2, n3 = 0;
    int i, j;
    // Input for Polynomial 1
    printf("Enter number of terms in Polynomial 1: ");
    scanf("%d", &n1);
```

```
printf("Enter terms (coefficient exponent) in decreasing order of exponent:\n");
for (i = 0; i < n1; i++) {
    scanf("%d%d", &poly1[i].coeff, &poly1[i].exp);
}
// Input for Polynomial 2
printf("Enter number of terms in Polynomial 2: ");
scanf("%d", &n2);
printf("Enter terms (coefficient exponent) in decreasing order of exponent:\n");
for (i = 0; i < n2; i++) {
    scanf("%d%d", &poly2[i].coeff, &poly2[i].exp);
}
i = j = 0;
// Merge-like addition
while (i < n1 && j < n2) {
    if (poly1[i].exp == poly2[j].exp) {
        result[n3].coeff = poly1[i].coeff + poly2[j].coeff;
        result[n3].exp = poly1[i].exp;
        i++; j++; n3++;
    }
    else if (poly1[i].exp > poly2[j].exp) {
        result[n3] = poly1[i];
        i++; n3++;
    }
    else {
        result[n3] = poly2[j];
        j++; n3++;
    }
}
// Remaining terms of poly1
while (i < n1) {
    result[n3++] = poly1[i++];
}
// Remaining terms of poly2
while (j < n2) {
```

```
    result[n3++] = poly2[j++];
}
// Display Result
printf("\nResultant Polynomial:\n");
for (i = 0; i < n3; i++) {
    if (result[i].coeff != 0)
        printf("%dx^%d ", result[i].coeff, result[i].exp);
    if (i < n3 - 1)
        printf("+ ");
}
printf("\n");
return 0;
}
```

OUTPUT

Enter number of terms in Polynomial 1:3

Enter terms (coefficient exponent) in decreasing order of exponent:

3 4

2 2

5 0

$3x^4 + 2x^2 + 5$

Enter number of terms in Polynomial 2:3

Enter terms (coefficient exponent) in decreasing order of exponent:

4 3

2 2

7 0

$4x^3 + 2x^2 + 7$

Resultant Polynomial:

$3x^4 + 4x^3 + 4x^2 + 12x^0$

MODEL VIVA QUESTIONS

1. Why should the polynomial terms be arranged in descending order of exponents?
2. What happens if the exponents are equal?
3. What is the time and space complexity of polynomial addition?

RESULT

The sum of the two sparse polynomials is successfully computed using arrays

EXP NO:2**TRANPOSE OF A SPARSE MATRIX AND ADDITION OF SPARSE MATRICES**

AIM: To Find the transpose of a sparse matrix and sum of two sparse matrices

LEARNING OUTCOMES

Students will be able to represent sparse matrices using the three-tuple format, find the transpose of a sparse matrix, perform the addition of two sparse matrices, and implement efficient algorithms to optimize memory usage and computation.

ALGORITHM**readSparse(sp)**

Input: Empty sparse matrix array sp[]

Output: Sparse matrix in 3-tuple form

1. Start
2. Read the number of rows $\rightarrow$ sp[0].row
3. Read the number of columns $\rightarrow$ sp[0].col
4. Read number of non-zero elements $\rightarrow$ sp[0].val
5. For i = 1 to sp[0].val
 - a. Read row, column, value
 - b. Store them in sp[i].row, sp[i].col, sp[i].val
6. Stop

printSparse(sp)

Input: Sparse matrix in 3-tuple format

Output: Printed sparse matrix

1. Start
2. Let n = sp[0].val
3. Print heading (Row, Col, Value)
4. For i = 0 to n
 - a. Print sp[i].row, sp[i].col, sp[i].val

5. Stop

transpose(a, b)

Input: Sparse matrix a[]

Output: Transpose matrix b[]

1. Start
2. Set:
 - $b[0].row = a[0].col$
 - $b[0].col = a[0].row$
 - $b[0].val = a[0].val$
3. Initialize $k = 1$
4. For each column index $i = 0$ to $a[0].col - 1$
 - a. For each non-zero entry $j = 1$ to $a[0].val$
 - i. If $a[j].col == i$
 - Set $b[k].row = a[j].col$
 - Set $b[k].col = a[j].row$
 - Set $b[k].val = a[j].val$
 - Increment k
5. Stop

addSparse(a, b, c)

Input: Sparse matrices A and B

Output: Sparse matrix $C = A + B$

1. Start
2. If dimensions of A and B differ
 - Print "Addition not possible"
 - Stop
3. Initialize pointers:
 - $i = 1$
 - $j = 1$
 - $k = 1$
4. Set $c[0].row = a[0].row$
5. Set $c[0].col = a[0].col$

6. Repeat while both matrices have elements:
 - a. If $a[i]$ and $b[j]$ have same (row, col):
 - Add values $\rightarrow c[k].val = a[i].val + b[j].val$
 - Copy row, col
 - Increment i, j, k
 - b. Else if $a[i]$ comes earlier in row-major order
 - Copy $a[i]$ to $c[k]$
 - Increment i, k
 - c. Else
 - Copy $b[j]$ to $c[k]$
 - Increment j, k
7. Copy remaining entries of A (if any)
8. Copy remaining entries of B (if any)
9. Set $c[0].val = k - 1$
10. Stop

main()

1. Start
2. Declare A, B, T, C as sparse matrix arrays
3. Print "Enter Sparse Matrix A"
4. Call readSparse(A)
5. Print "Enter Sparse Matrix B"
6. Call readSparse(B)
7. Call transpose(A, T)
8. Call addSparse(A, B, C)
9. Print A using printSparse(A)
10. Print B using printSparse(B)
11. Print T using printSparse(T)
12. Print C using printSparse(C)
13. Stop

PROGRAM

```
#include <stdio.h>
```

```
#define MAX 50
// Structure for sparse matrix term (row, column, value)
typedef struct {
    int row, col, val;
} Term;
// Function to read sparse matrix
void readSparse(Term sp[]) {
    int i, r, c;
    printf("Enter number of rows and columns: ");
    scanf("%d%d", &sp[0].row, &sp[0].col);

    printf("Enter number of non-zero elements: ");
    scanf("%d", &sp[0].val);

    printf("Enter row, column and value of each non-zero element:\n");
    for (i = 1; i <= sp[0].val; i++) {
        scanf("%d%d%d", &sp[i].row, &sp[i].col, &sp[i].val);
    }
}
// Function to print sparse matrix in 3-tuple form
void printSparse(Term sp[]) {
    int i, n = sp[0].val;
    printf("\nRow\tCol\tValue\n");
    for (i = 0; i <= n; i++) {
        printf("%d\t%d\t%d\n", sp[i].row, sp[i].col, sp[i].val);
    }
}
// Function to find transpose
void transpose(Term a[], Term b[]) {
    int i, j, n = a[0].val;

    b[0].row = a[0].col;
    b[0].col = a[0].row;
    b[0].val = n;
    int k = 1;
    for (i = 0; i < a[0].col; i++) {
        for (j = 1; j <= n; j++) {
            if (a[j].col == i) {
                b[k].row = a[j].col;
                b[k].col = a[j].row;
                b[k].val = a[j].val;
                k++;
            }
        }
    }
}
```

```
    }  
}  
// Function to add two sparse matrices  
void addSparse(Term a[], Term b[], Term c[]) {  
    if (a[0].row != b[0].row || a[0].col != b[0].col) {  
        printf("Addition not possible. Dimensions mismatch!\n");  
        return;  
    }  
    int i = 1, j = 1, k = 1;  
    int n1 = a[0].val, n2 = b[0].val;  
    c[0].row = a[0].row;  
    c[0].col = a[0].col;  
    while (i <= n1 && j <= n2) {  
        if (a[i].row == b[j].row && a[i].col == b[j].col) {  
            c[k].row = a[i].row;  
            c[k].col = a[i].col;  
            c[k].val = a[i].val + b[j].val;  
            i++; j++; k++;  
        }  
        else if (a[i].row < b[j].row ||  
            (a[i].row == b[j].row && a[i].col < b[j].col)) {  
            c[k++] = a[i++];  
        }  
        else {  
            c[k++] = b[j++];  
        }  
    }  
    while (i <= n1) c[k++] = a[i++];  
    while (j <= n2) c[k++] = b[j++];  
    c[0].val = k - 1;  
}  
int main() {  
    Term A[MAX], B[MAX], T[MAX], C[MAX];  
  
    // Read matrix A  
    printf("\nEnter Sparse Matrix A\n");  
    readSparse(A);  
  
    // Read matrix B  
    printf("\nEnter Sparse Matrix B\n");  
    readSparse(B);  
  
    // Transpose of A  
    transpose(A, T);
```

```
// Addition of A and B
addSparse(A, B, C);
// Display results
printf("\n--- Sparse Matrix A ---");
printSparse(A);
printf("\n--- Sparse Matrix B ---");
printSparse(B);
printf("\n--- Transpose of A ---");
printSparse(T);
printf("\n--- Sum A + B ---");
printSparse(C);
return 0;
}
```

OUTPUT

Enter Sparse Matrix A
Enter number of rows and columns: 3 3
Enter number of non-zero elements: 3
Enter row, column and value of each non-zero element:
0 2 5
1 1 3
2 0 4

Enter Sparse Matrix B
Enter number of rows and columns: 3 3
Enter number of non-zero elements: 3
Enter row, column and value of each non-zero element:
0 1 6
1 1 2
2 2 7

--- Sparse Matrix A ---

Row	Col	Value
-----	-----	-------

3	3	3
0	2	5
1	1	3
2	0	4

--- Sparse Matrix B ---

Row	Col	Value
-----	-----	-------

3	3	3
---	---	---

0	1	6
1	1	2
2	2	7

--- Transpose of A ---

Row	Col	Value
3	3	3
2	0	5
1	1	3
0	2	4

--- Sum A + B ---

Row	Col	Value
3	3	4
0	1	6
0	2	5
1	1	5
2	0	4
2	2	7

MODEL VIVA QUESTIONS

1. What is a sparse matrix?
2. Why do we use sparse matrix representation?
3. What is the difference between a sparse matrix and a dense matrix?
4. What are the advantages of storing sparse matrices using arrays?
5. What are the different methods of representing sparse matrices?

RESULT

The transpose of the sparse matrix and the addition of two sparse matrices are successfully performed using array representation.

EXP NO:3**INFIX EXPRESSION TO POSTFIX (OR PREFIX) AND EVALUATION**

AIM: To Convert infix expression to postfix (or prefix) and then evaluate using stack

LEARNING OUTCOMES

Students will be able to convert infix expressions into postfix or prefix notation using stacks, evaluate postfix or prefix expressions efficiently, and implement stack-based algorithms for expression processing.

ALGORITHM

1. Start
2. Declare infix and postfix strings
3. Read infix expression from user
4. Call **infixToPostfix()** to convert infix to postfix

Initialize an empty stack

Scan the infix expression from left to right

For each character **ch** in the infix expression:

Case 1: If **ch** is an operand

→ Append it to the postfix expression

Case 2: If **ch** is '('

→ Push it onto the stack

Case 3: If **ch** is ')'

→ Repeatedly pop from the stack and append to postfix

→ Stop when '(' is encountered

→ Remove '(' from stack

Case 4: If **ch** is an operator

→ While the stack is not empty **AND**

$\text{precedence}(\text{top of stack}) \geq \text{precedence}(\text{ch})$

→ Pop from stack and append to postfix

→ Push **ch** onto the stack

After the full expression is scanned:

→ Pop all remaining operators from the stack into postfix

Add NULL terminator to postfix

5. Print the postfix expression
6. Call **evaluatePostfix()**
Initialize empty evaluation stack
Scan the postfix expression from left to right
For each symbol **ch**:
Case 1: If **ch** is a digit
→ Convert character to number
→ Push it onto evaluation stack
Case 2: If **ch** is an operator
→ Pop operand2
→ Pop operand1
→ Apply the operator:
+, -, *, /, ^
→ Push result back onto stack
After scanning ends:
→ Pop the final value from stack → This is the result
7. Print the evaluated result
8. Stop

PROGRAM

```
#include <stdio.h>
#include <ctype.h>
#include <string.h>
#define MAX 100
char stack[MAX];
int top = -1;
void push(char x) {
    stack[++top] = x;
}
char pop() {
    if (top == -1)
        return -1;
    else
        return stack[top--];
}
```

```
}  
int precedence(char x) {  
    if (x == '(') return 0;  
    if (x == '+' || x == '-') return 1;  
    if (x == '*' || x == '/') return 2;  
    if (x == '^') return 3;  
    return 0;  
}  
  
int isOperator(char x) {  
    return (x=='+' || x=='-' || x=='*' || x=='/' || x=='^');  
}  
  
// ----- INFIX TO POSTFIX -----  
void infixToPostfix(char infix[], char postfix[]) {  
    int i, j = 0;  
    char x;  
  
    for (i = 0; infix[i] != '\0'; i++) {  
        if (isalnum(infix[i])) {  
            postfix[j++] = infix[i];  
        }  
        else if (infix[i] == '(') {  
            push(infix[i]);  
        }  
        else if (infix[i] == ')') {  
            while ((x = pop()) != '(')  
                postfix[j++] = x;  
        }  
        else {  
            while (precedence(stack[top]) >= precedence(infix[i]))  
                postfix[j++] = pop();  
            push(infix[i]);  
        }  
    }  
}
```

```
while (top != -1)
    postfix[j++] = pop();

postfix[j] = '\0';
}

// ----- POSTFIX EVALUATION -----

int evalStack[MAX];
int etop = -1;
void epush(int x) {
    evalStack[++etop] = x;
}
int epop() {
    return evalStack[etop--];
}
int evaluatePostfix(char postfix[]) {
    int i, op1, op2;
    for (i = 0; postfix[i] != '\0'; i++) {
        if (isdigit(postfix[i])) {
            epush(postfix[i] - '0'); // convert char to int
        }
        else if (isOperator(postfix[i])) {
            op2 = epop();
            op1 = epop();

            switch (postfix[i]) {
                case '+': epush(op1 + op2); break;
                case '-': epush(op1 - op2); break;
                case '*': epush(op1 * op2); break;
                case '/': epush(op1 / op2); break;
                case '^': {
                    int p = 1;
                    for (int k = 0; k < op2; k++) p *= op1;
                    epush(p);
                    break;
                }
            }
        }
    }
    return evalStack[etop];
}
```

```
    }
    }
}
return epop();
}

// ----- MAIN -----
int main() {
    char infix[MAX], postfix[MAX];
    printf("Enter infix expression: ");
    scanf("%s", infix);
    infixToPostfix(infix, postfix);
    printf("\nPostfix Expression: %s\n", postfix);
    int result = evaluatePostfix(postfix);
    printf("Evaluated Result: %d\n", result);
    return 0;
}
```

OUTPUT

Enter infix expression: 5*(6-2)
Postfix Expression: 562-*
Evaluated Result: 20

Enter infix expression: 8/4+3*2
Postfix Expression: 84/32*+
Evaluated Result: 8

MODEL VIVA QUESTIONS

1. What is an infix expression?
2. What is a prefix expression?
3. Why do we convert infix expressions into postfix or prefix form?
4. What is the advantage of postfix notation?
5. What is operator precedence?

6. What is operator associativity?

RESULT

The given infix expression is successfully converted into postfix (or prefix) notation using a stack. The converted expression is then evaluated using stack operations, and the correct result is obtained successfully.

EXP NO:4**QUEUE, DEQUEUE, AND CIRCULAR QUEUE USING ARRAY**

AIM: To Implement Queue, DEQUEUE, and Circular Queue using arrays.

LEARNING OUTCOMES

Students will be able to implement Queue, Dequeue (Double-Ended Queue), and Circular Queue using arrays, perform insertion and deletion operations efficiently, and apply these data structures to solve FIFO-based problems.

ALGORITHM**LINEAR QUEUE USING ARRAYS**

1. Repeat forever:
2. Display menu: Enqueue, Dequeue, Display, Exit
3. Read choice
4. If choice = 1 → Read x and call ENQUEUE(x)
 - If rear == MAX – 1, print **Overflow** and return.
 - If front == -1, set front = 0.
 - Increment rear.
 - Set queue[rear] = x.
 - Print that x is inserted
5. If choice = 2 → Call DEQUEUE()
 - If front == -1 or front > rear, print **Underflow** and return.
 - Print the element queue[front].
 - Increment front
6. If choice = 3 → Call DISPLAY()
 - If front == -1 or front > rear, print **Queue empty** and return.
 - For i = front to rear, print queue[i]
7. If choice = 4 → Stop program
8. Else print **Invalid choice**

DEQUEUE (DOUBLE-ENDED QUEUE)

1. Repeat forever:
2. Display menu

3. Read user choice
4. if choice = 1 → Read x and call insertFront(x)
If deque is full
Condition: (front == 0 AND rear == MAX-1) OR (front == rear + 1)
→ Print **Deque Full** and return.
If deque is empty (front == -1)
→ Set front = rear = 0.
Else if front == 0
→ Set front = MAX - 1.
Else
→ Decrement front.
Store x at deque[front]
5. If choice = 2 → Read x and call insertRear(x)
If deque is full
→ Print **Deque Full** and return.
If deque is empty
→ Set front = rear = 0.
Else if rear == MAX - 1
→ Set rear = 0.
Else
→ Increment rear.
Store x at deque[rear]
6. If choice = 3 → Call deleteFront()
If deque is empty (front == -1)
→ Print **Deque Empty** and return.
Print element at deque[front].
If front == rear
→ Set front = rear = -1.
Else if front == MAX - 1
→ Set front = 0.
Else
→ Increment front.

7. If choice = 4 → Call deleteRear()
If deque is empty
→ Print **Deque Empty** and return.
Print element at deque[rear].
If front == rear
→ Set front = rear = -1.
Else if rear == 0
→ Set rear = MAX - 1.
Else
→ Decrement rear
8. If choice = 5 → Call display()
If deque is empty
→ Print **Deque empty** and return.
Set i = front.
Repeat:
Print deque[i].
Set i = (i + 1) % MAX.
Until i == rear + 1 (mod MAX)
9. If choice = 6 → Exit program
10. Else print Invalid choice

CIRCULAR QUEUE

1. Repeat forever:
2. Display menu:
3. Read user choice
4. If choice = 1 → Read x and call ENQUEUE(x)
Check if queue is full:
Condition: (front == 0 AND rear == MAX-1) OR (rear + 1) % MAX == front
→ If TRUE, print Circular Queue Overflow and return.
If queue is empty (front == -1)
→ Set front = rear = 0.
Else
→ Update rear = (rear + 1) % MAX.
Insert element: cqueue[rear] = x.
Print x inserted.

5. If choice = 2 → Call DEQUEUE()
Check if queue is empty (front == -1)
→ If TRUE, print Circular Queue Underflow and return.
Print element at front: cqueue[front].
If front == rear
→ Only one element → Set front = rear = -1.
Else
→ Update front = (front + 1) % MAX.
6. If choice = 3 → Call DISPLAY()
Check if queue is empty (front == -1)
→ If TRUE, print Circular Queue is empty and return.
Set i = front.
Repeat:
Print cqueue[i]
If i == rear, stop
Else i = (i + 1) % MAX
7. If choice = 4 → Exit program
8. Else → Print Invalid choice

PROGRAM

LINEAR QUEUE USING ARRAYS

```
#include <stdio.h>
#define MAX 5
int queue[MAX];
int front = -1, rear = -1;

void enqueue(int x) {
    if (rear == MAX - 1)
        printf("Queue Overflow!\n");
    else {
        if (front == -1) front = 0;
        queue[++rear] = x;
        printf("%d inserted.\n", x);
    }
}
```

```
    }  
}  
void dequeue() {  
    if (front == -1 || front > rear)  
        printf("Queue Underflow!\n");  
    else  
        printf("%d deleted.\n", queue[front++]);  
}  
void display() {  
    if (front == -1 || front > rear)  
        printf("Queue is empty.\n");  
    else {  
        printf("Queue: ");  
        for (int i = front; i <= rear; i++)  
            printf("%d ", queue[i]);  
        printf("\n");  
    }  
}  
int main() {  
    int choice, x;  
    while (1) {  
        printf("\n--- LINEAR QUEUE MENU ---\n");  
        printf("1. Enqueue\n2. Dequeue\n3. Display\n4. Exit\n");  
        printf("Enter choice: ");  
        scanf("%d", &choice);  
  
        switch (choice) {  
            case 1: printf("Enter value: "); scanf("%d", &x); enqueue(x); break;  
            case 2: dequeue(); break;  
            case 3: display(); break;  
            case 4: return 0;  
            default: printf("Invalid choice!\n");  
        }  
    }  
}
```

```
}
```

DEQUEUE (DOUBLE-ENDED QUEUE)

```
#include <stdio.h>
```

```
#define MAX 5
```

```
int deque[MAX];
```

```
int front = -1, rear = -1;
```

```
void insertFront(int x) {
```

```
    if ((front == 0 && rear == MAX - 1) || (front == rear + 1))
```

```
        printf("Deque is Full!\n");
```

```
    else {
```

```
        if (front == -1)
```

```
            front = rear = 0;
```

```
        else if (front == 0)
```

```
            front = MAX - 1;
```

```
        else
```

```
            front--;
```

```
        deque[front] = x;
```

```
        printf("%d inserted at front.\n", x);
```

```
    }
```

```
}
```

```
void insertRear(int x) {
```

```
    if ((front == 0 && rear == MAX - 1) || (front == rear + 1))
```

```
        printf("Deque is Full!\n");
```

```
    else {
```

```
        if (rear == -1)
```

```
            front = rear = 0;
```

```
        else if (rear == MAX - 1)
```

```
            rear = 0;
```

```
        else
```

```
            rear++;
```

```
    deque[rear] = x;
    printf("%d inserted at rear.\n", x);
}
}

void deleteFront() {
    if (front == -1)
        printf("Deque is Empty!\n");
    else {
        printf("%d deleted from front.\n", deque[front]);
        if (front == rear)
            front = rear = -1;
        else if (front == MAX - 1)
            front = 0;
        else
            front++;
    }
}

void deleteRear() {
    if (rear == -1)
        printf("Deque is Empty!\n");
    else {
        printf("%d deleted from rear.\n", deque[rear]);
        if (front == rear)
            front = rear = -1;
        else if (rear == 0)
            rear = MAX - 1;
        else
            rear--;
    }
}

void display() {
    if (front == -1)
```

```
    printf("Deque is empty.\n");
else {
    printf("Deque: ");
    int i = front;
    while (1) {
        printf("%d ", deque[i]);
        if (i == rear) break;
        i = (i + 1) % MAX;
    }
    printf("\n");
}
}

int main() {
    int choice, x;
    while (1) {
        printf("\n--- DEQUEUE MENU ---\n");
        printf("1. Insert Front\n2. Insert Rear\n3. Delete Front\n4. Delete Rear\n5. Display\n6. Exit\n");
        scanf("%d", &choice);
        switch (choice) {
            case 1: printf("Enter value: "); scanf("%d", &x); insertFront(x); break;
            case 2: printf("Enter value: "); scanf("%d", &x); insertRear(x); break;
            case 3: deleteFront(); break;
            case 4: deleteRear(); break;
            case 5: display(); break;
            case 6: return 0;
            default: printf("Invalid choice!\n");
        }
    }
}
```

CIRCULAR QUEUE

```
#include <stdio.h>

#define MAX 5

int cqueue[MAX];
```

```
int front = -1, rear = -1;

void enqueue(int x) {
    if ((front == 0 && rear == MAX - 1) || (rear + 1) % MAX == front)
        printf("Circular Queue Overflow!\n");
    else {
        if (front == -1)
            front = rear = 0;
        else
            rear = (rear + 1) % MAX;
        cqueue[rear] = x;
        printf("%d inserted.\n", x);
    }
}

void dequeue() {
    if (front == -1)
        printf("Circular Queue Underflow!\n");
    else {
        printf("%d deleted.\n", cqueue[front]);
        if (front == rear)
            front = rear = -1;
        else
            front = (front + 1) % MAX;
    }
}

void display() {
    if (front == -1)
        printf("Circular Queue is empty.\n");
    else {
        printf("Circular Queue: ");
        int i = front;
        while (1) {
            printf("%d ", cqueue[i]);
            if (i == rear) break;
        }
    }
}
```

```
        i = (i + 1) % MAX;
    }
    printf("\n");
}
}
int main() {
    int choice, x;
    while (1) {
        printf("\n--- CIRCULAR QUEUE MENU ---\n");
        printf("1. Enqueue\n2. Dequeue\n3. Display\n4. Exit\n");
        scanf("%d", &choice);
        switch (choice) {
            case 1: printf("Enter value: "); scanf("%d", &x); enqueue(x); break;
            case 2: dequeue(); break;
            case 3: display(); break;
            case 4: return 0;
            default: printf("Invalid choice!\n");
        }
    }
}
```

OUTPUT

LINEAR QUEUE USING ARRAYS

--- LINEAR QUEUE MENU ---

1. Enqueue

2. Dequeue

3. Display

4. Exit

Enter choice: 1

Enter value: 10

10 inserted.

--- LINEAR QUEUE MENU ---

1. Enqueue

2. Dequeue

3. Display

4. Exit

Enter choice: 1

Enter value: 20

20 inserted.

--- LINEAR QUEUE MENU ---

1. Enqueue

2. Dequeue

3. Display

4. Exit

Enter choice: 1

Enter value: 30

30 inserted.

--- LINEAR QUEUE MENU ---

1. Enqueue

2. Dequeue

3. Display

4. Exit

Enter choice: 3

Queue: 10 20 30

--- LINEAR QUEUE MENU ---

1. Enqueue

2. Dequeue

3. Display

4. Exit

Enter choice: 2

10 deleted.

--- LINEAR QUEUE MENU ---

1. Enqueue

2. Dequeue

3. Display

4. Exit

Enter choice: 3

Queue: 20 30

--- LINEAR QUEUE MENU ---

1. Enqueue

2. Dequeue

3. Display

4. Exit

Enter choice: 1

Enter value: 40

40 inserted.

--- LINEAR QUEUE MENU ---

1. Enqueue

2. Dequeue

3. Display

4. Exit

Enter choice: 1

Enter value: 50

50 inserted.

--- LINEAR QUEUE MENU ---

1. Enqueue

2. Dequeue

3. Display

4. Exit

Enter choice: 1

Enter value: 60

Queue Overflow!

--- LINEAR QUEUE MENU ---

1. Enqueue

2. Dequeue

3. Display

4. Exit

Enter choice: 3

Queue: 20 30 40 50

--- LINEAR QUEUE MENU ---

1. Enqueue

2. Dequeue

3. Display

4. Exit

Enter choice: 4

DEQUEUE (DOUBLE-ENDED QUEUE)

--- DEQUEUE MENU ---

1. Insert Front

2. Insert Rear

3. Delete Front

4. Delete Rear

5. Display

6. Exit

Enter choice: 2

Enter value: 10

10 inserted at rear.

--- DEQUEUE MENU ---

1. Insert Front

2. Insert Rear

3. Delete Front

4. Delete Rear

5. Display

6. Exit

Enter choice: 2

Enter value: 20

20 inserted at rear.

--- DEQUEUE MENU ---

1. Insert Front
2. Insert Rear
3. Delete Front
4. Delete Rear
5. Display
6. Exit

Enter choice: 1

Enter value: 5

5 inserted at front.

--- DEQUEUE MENU ---

1. Insert Front
2. Insert Rear
3. Delete Front
4. Delete Rear
5. Display
6. Exit

Enter choice: 5

Deque: 5 10 20

--- DEQUEUE MENU ---

1. Insert Front
2. Insert Rear
3. Delete Front
4. Delete Rear
5. Display
6. Exit

Enter choice: 3

5 deleted from front.

--- DEQUEUE MENU ---

1. Insert Front
2. Insert Rear
3. Delete Front
4. Delete Rear
5. Display
6. Exit

Enter choice: 4

20 deleted from rear.

--- DEQUEUE MENU ---

1. Insert Front
2. Insert Rear
3. Delete Front
4. Delete Rear
5. Display
6. Exit

Enter choice: 5

Deque: 10

--- DEQUEUE MENU ---

1. Insert Front
2. Insert Rear
3. Delete Front
4. Delete Rear
5. Display
6. Exit

Enter choice: 6

CIRCULAR QUEUE

--- CIRCULAR QUEUE MENU ---

1. Enqueue
2. Dequeue
3. Display
4. Exit

Enter choice: 1

Enter value: 10

10 inserted.

--- CIRCULAR QUEUE MENU ---

1. Enqueue

2. Dequeue

3. Display

4. Exit

Enter choice: 1

Enter value: 20

20 inserted.

--- CIRCULAR QUEUE MENU ---

1. Enqueue

2. Dequeue

3. Display

4. Exit

Enter choice: 1

Enter value: 30

30 inserted.

--- CIRCULAR QUEUE MENU ---

1. Enqueue

2. Dequeue

3. Display

4. Exit

Enter choice: 3

Circular Queue: 10 20 30

--- CIRCULAR QUEUE MENU ---

1. Enqueue

2. Dequeue

3. Display

4. Exit

Enter choice: 2

10 deleted.

--- CIRCULAR QUEUE MENU ---

1. Enqueue

2. Dequeue

3. Display

4. Exit

Enter choice: 1

Enter value: 40

40 inserted.

--- CIRCULAR QUEUE MENU ---

1. Enqueue

2. Dequeue

3. Display

4. Exit

Enter choice: 3

Circular Queue: 20 30 40

--- CIRCULAR QUEUE MENU ---

1. Enqueue

2. Dequeue

3. Display

4. Exit

Enter choice: 1

Enter value: 50

50 inserted.

--- CIRCULAR QUEUE MENU ---

1. Enqueue

2. Dequeue

3. Display

4. Exit

Enter choice: 1

Enter value: 60

Circular Queue Overflow!

--- CIRCULAR QUEUE MENU ---

1. Enqueue

2. Dequeue

3. Display

4. Exit

Enter choice: 3

Circular Queue: 20 30 40 50

--- CIRCULAR QUEUE MENU ---

1. Enqueue

2. Dequeue

3. Display

4. Exit

MODEL QUESTIONS

1. What is a queue?
2. Which principle does a queue follow?
3. What are the basic operations of a queue?
4. What is the difference between enqueue and dequeue?
5. What are the applications of a queue?

RESULT

The Queue, Dequeue (Double-Ended Queue), and Circular Queue operations are successfully implemented using arrays. The insertion, deletion, and display operations are performed correctly, and the circular queue efficiently reuses vacant array positions.

EXP NO:5**BROWSER NAVIGATION USING DOUBLY LINKED LIST**

AIM: Implement backward and forward navigation of visited web pages in a web browser (i.e.back and forward buttons) using doubly linked list operations.

LEARNING OUTCOMES

By the end of this experiment, students will be able to implement backward and forward navigation of visited web pages using doubly linked list operations and understand the practical application of doubly linked lists in browser history management.

ALGORITHM

1. Repeat forever:
2. Display menu:
 - a. Visit new page
 - b. Back
 - c. Forward
 - d. Display History
 - e. Exit
3. Read user choice
4. If choice = 1 → Read URL and call VISIT NEW PAGE(url).

Create a new node newNode with url.

Set newNode->prev = current and newNode->next = NULL.

If current is not NULL (some page is currently open):

- a. Delete all nodes in the forward history (current->next).
- b. Set current->next = newNode.

If current is NULL (no pages visited yet):

- a. Set head = newNode.

Update current = newNode.

Print Visited: url.

5. If choice = 2 → Call GO BACK().

6. If `current == NULL` OR `current->prev == NULL`

→ Print No previous page.

Else

→ Move `current = current->prev`.

→ Print Current page: `current->url`.

7. If `choice = 3` → Call `GO FORWARD()`

If `current == NULL` OR `current->next == NULL`

→ Print No forward page.

Else

→ Move `current = current->next`.

→ Print Current page: `current->url`.

8. If. If `choice = 4` → Call `DISPLAY HISTORY()`.

Set `temp = head`.

While `temp != NULL`:

a. If `temp == current` → Print [`temp->url`] <- Current.

b. Else → Print `temp->url`.

c. Move `temp = temp->next`.

9. If `choice = 5` → Exit program.

10. Else → Print Invalid choice.

PROGRAM

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
// Node structure for each visited page
```

```
typedef struct Node {
```

```
    char url[100];
```

```
    struct Node* prev;
```

```
    struct Node* next;
```

```
} Node;
```

```
Node* head = NULL; // Start of the list
Node* current = NULL; // Current page

// Function to visit a new page
void visitPage(char url[]) {
    Node* newNode = (Node*)malloc(sizeof(Node));
    strcpy(newNode->url, url);
    newNode->prev = current;
    newNode->next = NULL;

    if (current != NULL) {
        // Clear forward history
        Node* temp = current->next;
        while (temp != NULL) {
            Node* t = temp;
            temp = temp->next;
            free(t);
        }
        current->next = newNode;
    } else {
        head = newNode;
    }

    current = newNode;
    printf("Visited: %s\n", url);
}

// Function to go back
void goBack() {
    if (current == NULL || current->prev == NULL) {
        printf("No previous page!\n");
    } else {
        current = current->prev;
    }
}
```

```
    printf("Current page: %s\n", current->url);
}
}

// Function to go forward
void goForward() {
    if (current == NULL || current->next == NULL) {
        printf("No forward page!\n");
    } else {
        current = current->next;
        printf("Current page: %s\n", current->url);
    }
}

// Function to display browser history
void displayHistory() {
    Node* temp = head;
    printf("Browser History:\n");
    while (temp != NULL) {
        if (temp == current)
            printf("[ %s ] <- Current\n", temp->url);
        else
            printf("%s\n", temp->url);
        temp = temp->next;
    }
}

int main() {
    int choice;
    char url[100];

    while (1) {
        printf("\n--- Browser Navigation Menu ---\n");
        printf("1. Visit new page\n2. Back\n3. Forward\n4. Display History\n5. Exit\n");
```

```
printf("Enter choice: ");
scanf("%d", &choice);
getchar(); // clear newline

switch (choice) {
    case 1:
        printf("Enter URL: ");
        fgets(url, 100, stdin);
        url[strcspn(url, "\n")] = 0; // remove newline
        visitPage(url);
        break;
    case 2:
        goBack();
        break;
    case 3:
        goForward();
        break;
    case 4:
        displayHistory();
        break;
    case 5:
        printf("Exiting browser...\n");
        exit(0);
    default:
        printf("Invalid choice!\n");
}
}
return 0;
}
```

OUTPUT

--- Browser Navigation Menu ---

1. Visit new page
2. Back
3. Forward

4. Display History

5. Exit

Enter choice: 1

Enter URL: google.com

Visited: google.com

Enter choice: 1

Enter URL: facebook.com

Visited: facebook.com

Enter choice: 2

Current page: google.com

Enter choice: 3

Current page: facebook.com

Enter choice: 4

Browser History:

google.com

[facebook.com] <- Current

MODEL VIVA QUESTIONS

1. What is a doubly linked list?
2. How is a doubly linked list different from a singly linked list?
3. What are the components of a doubly linked list node?
4. Why is a doubly linked list used for browser navigation?
5. What are the advantages of a doubly linked list?

RESULT

The backward and forward navigation of visited web pages is successfully implemented using a doubly linked list. The Back and Forward operations work correctly by moving through the previous and next nodes, enabling efficient navigation between visited web pages.

EXP NO:6**POLYNOMIAL ADDITION AND MULTIPLICATION USING SINGLY LINKED LIST**

AIM: Implement addition and multiplication of polynomials using singly linked lists

LEARNING OUTCOME

By the end of this experiment, students will be able to implement polynomial addition and multiplication using singly linked lists and apply linked list operations to efficiently represent and manipulate polynomial expressions.

ALGORITHM

1. Start
2. Create two polynomials poly1 and poly2 using CREATE POLYNOMIAL.
3. Display poly1 and poly2.
4. Call ADD POLYNOMIALS(poly1, poly2) → store in sum.
5. Display sum.
6. Call MULTIPLY POLYNOMIALS(poly1, poly2) → store in product.
7. Display product

Node Structure

- Each node represents a term of the polynomial:
 - coeff → coefficient of term
 - exp → exponent of term
 - next → pointer to next term
- Maintain **head pointer** for each polynomial.

INSERT TERM (poly, coeff, exp)

1. Create a new node newNode with coeff and exp.
If list is empty (poly == NULL)
→ Make newNode the head of list.
2. Else, traverse the list to find correct position based on **descending exponent**.
3. If a node with same exponent exists

→ Add coefficients (existing.coeff += coeff).

4. Else

→ Insert newNode at proper position.

ADDITION OF TWO POLYNOMIALS (poly1, poly2)

1. Initialize result = NULL.
2. Traverse poly1, for each term:
→ INSERT TERM(result, coeff, exp).
3. Traverse poly2, for each term:
→ INSERT TERM(result, coeff, exp).
4. Return result.

MULTIPLICATION OF TWO POLYNOMIALS (poly1, poly2)

1. Initialize result = NULL.
2. For each term t1 in poly1:
 - a. For each term t2 in poly2:
→ Multiply coefficients: $c = t1.coeff * t2.coeff$
→ Add exponents: $e = t1.exp + t2.exp$
→ INSERT TERM(result, c, e).
3. Return result.

DISPLAY POLYNOMIAL (poly)

1. If poly == NULL
→ Print 0 and return.
2. Traverse the list:
→ Print each term as coeff x^exp.
3. Separate terms with +.

CREATE POLYNOMIAL

1. Read number of terms n.
2. Repeat n times:
→ Read coeff and exp.

→ Call INSERT TERM(poly, coeff, exp).

PROGRAM

```
#include <stdio.h>
#include <stdlib.h>

// Node structure for polynomial term
typedef struct Node {
    int coeff;
    int exp;
    struct Node* next;
} Node;

// Function to create a new term
Node* createNode(int coeff, int exp) {
    Node* newNode = (Node*)malloc(sizeof(Node));
    newNode->coeff = coeff;
    newNode->exp = exp;
    newNode->next = NULL;
    return newNode;
}

// Function to insert term in descending order of exponent
void insertTerm(Node** poly, int coeff, int exp) {
    Node* newNode = createNode(coeff, exp);
    if (*poly == NULL || exp > (*poly)->exp) {
        newNode->next = *poly;
        *poly = newNode;
        return;
    }
    Node* temp = *poly;
    while (temp->next != NULL && temp->next->exp >= exp) {
        if (temp->next->exp == exp) { // Same exponent, add coefficients
            temp->next->coeff += coeff;
            free(newNode);
            return;
        }
    }
}
```

```
    temp = temp->next;
}
newNode->next = temp->next;
temp->next = newNode;
}

// Function to display polynomial
void displayPoly(Node* poly) {
    if (poly == NULL) {
        printf("0\n");
        return;
    }
    Node* temp = poly;
    while (temp != NULL) {
        printf("%dx^%d", temp->coeff, temp->exp);
        temp = temp->next;
        if (temp != NULL) printf(" + ");
    }
    printf("\n");
}

// Function to add two polynomials
Node* addPoly(Node* p1, Node* p2) {
    Node* result = NULL;
    while (p1 != NULL) {
        insertTerm(&result, p1->coeff, p1->exp);
        p1 = p1->next;
    }
    while (p2 != NULL) {
        insertTerm(&result, p2->coeff, p2->exp);
        p2 = p2->next;
    }
    return result;
}
```

```
// Function to multiply two polynomials
Node* multiplyPoly(Node* p1, Node* p2) {
    Node* result = NULL;
    for (Node* t1 = p1; t1 != NULL; t1 = t1->next) {
        for (Node* t2 = p2; t2 != NULL; t2 = t2->next) {
            insertTerm(&result, t1->coeff * t2->coeff, t1->exp + t2->exp);
        }
    }
    return result;
}

// Function to create polynomial by user input
Node* createPolynomial() {
    int n, coeff, exp;
    Node* poly = NULL;
    printf("Enter number of terms: ");
    scanf("%d", &n);
    for (int i = 0; i < n; i++) {
        printf("Enter coefficient and exponent for term %d: ", i+1);
        scanf("%d %d", &coeff, &exp);
        insertTerm(&poly, coeff, exp);
    }
    return poly;
}

// Main function
int main() {
    printf("Polynomial 1:\n");
    Node* poly1 = createPolynomial();
    printf("Polynomial 2:\n");
    Node* poly2 = createPolynomial();

    printf("\nPolynomial 1: ");
```

```
displayPoly(poly1);
printf("Polynomial 2: ");
displayPoly(poly2);
Node* sum = addPoly(poly1, poly2);
printf("\nSum: ");
displayPoly(sum);
Node* product = multiplyPoly(poly1, poly2);
printf("Product: ");
displayPoly(product);
return 0;
}
```

OUTPUT

Polynomial 1:

Number of terms: 3

Terms: (3 2), (2 1), (1 0)

Polynomial 2:

Number of terms: 2

Terms: (4 1), (5 0)

Polynomial 1: $3x^2 + 2x^1 + 1x^0$

Polynomial 2: $4x^1 + 5x^0$

Sum: $3x^2 + 6x^1 + 6x^0$

Product: $12x^3 + 23x^2 + 14x^1 + 5x^0$

MODEL VIVA QUESTIONS

1. What is a singly linked list?
2. What are the components of a singly linked list node?
3. How is a singly linked list different from a doubly linked list?
4. What are the advantages of a singly linked list?
5. What is dynamic memory allocation?
6. Which functions are used for dynamic memory allocation in C?

RESULT

The addition and multiplication of two polynomials are successfully implemented using a singly linked list. The polynomial terms are stored as linked list nodes, and the operations are performed correctly by combining like terms and generating the resultant polynomial.

EXP NO:7**BINARY EXPRESSION TREE WITH PREFIX/POSTFIX CONVERSION**

AIM: To Create a binary tree for a given simple arithmetic expression and find the prefix / postfix equivalent

LEARNING OUTCOME

By the end of this experiment, students will be able to construct a binary expression tree for a given arithmetic expression and generate its equivalent prefix and postfix expressions through tree traversal techniques.

ALGORITHM**Node Structure**

- Each node contains:
 1. data → operand or operator
 2. left → pointer to left child
 3. right → pointer to right child

- Maintain **two stacks**:
 1. **Operators stack** → stores operators and parentheses
 2. **Operands stack** → stores tree nodes

- 1. Read fully parenthesized infix expression.
- 2. Call **INFIX TO EXPRESSION TREE** → get root node.

- 3. Initialize empty stacks: operators and operands.

Traverse the infix expression character by character:

- If character is '(' → push onto operators stack.
- If character is **operand** (alphanumeric) → create a node and push onto operands stack.
- If character is **operator** (+ - * /):
 - While top of operators stack has **higher or equal precedence** and is not '(':
 - Pop operator from operators stack.
 - Pop two nodes from operands stack.

- Make operator node as root, left child = first popped node, right child = second popped node.
 - Push this operator node onto operands stack.
 - Push current operator onto operators stack.
- If character is ')':
 - While top of operators stack is not '(':
 - Pop operator and create node as above.
 - Pop '(' from operators stack.

After traversal, while operators stack is not empty:

- Pop operator and create node as above.

The node remaining on operands stack is the **root** of the expression tree.

4. Print "Prefix expression: " and call **PREORDER(root)**.
 - Start at root node.
 - Print data.
 - Recursively traverse left subtree.
 - Recursively traverse right subtree.
5. Print "Postfix expression: " and call **POSTORDER(root)**.
 - Start at root node.
 - Recursively traverse left subtree.
 - Recursively traverse right subtree.
 - Print data.

PROGRAM

```
#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>
#include <string.h>
```

```
// Node structure for binary expression tree
typedef struct Node {
```

```
char data;
struct Node *left, *right;
} Node;

// Stack for tree nodes
typedef struct Stack {
    Node* items[100];
    int top;
} Stack;

// Stack operations
void initStack(Stack *s) { s->top = -1; }
int isEmpty(Stack *s) { return s->top == -1; }
void push(Stack *s, Node* node) { s->items[++(s->top)] = node; }
Node* pop(Stack *s) { return s->items[(s->top)--]; }
Node* peek(Stack *s) { return s->items[s->top]; }

// Function to create a new tree node
Node* createNode(char data) {
    Node* node = (Node*)malloc(sizeof(Node));
    node->data = data;
    node->left = node->right = NULL;
    return node;
}

// Function to return precedence of operators
int precedence(char op) {
    if (op == '+' || op == '-') return 1;
    if (op == '*' || op == '/') return 2;
    return 0;
}

// Check if character is operator
int isOperator(char ch) {
    return ch == '+' || ch == '-' || ch == '*' || ch == '/';
}
```

```
}

// Convert infix to binary expression tree
Node* infixToExprTree(char* exp) {
    Stack operators, operands;
    initStack(&operators);
    initStack(&operands);
    int i;
    for (i = 0; i < strlen(exp); i++) {
        char ch = exp[i];
        if (ch == ' ' || ch == '\t') continue;
        if (ch == '(') {
            push(&operators, createNode(ch));
        } else if (isdigit(ch)) {
            push(&operands, createNode(ch));
        } else if (isOperator(ch)) {
            while (!isEmpty(&operators) && peek(&operators)->data != '(' &&
                precedence(peek(&operators)->data) >= precedence(ch)) {
                Node* op = pop(&operators);
                op->right = pop(&operands);
                op->left = pop(&operands);
                push(&operands, op);
            }
            push(&operators, createNode(ch));
        } else if (ch == ')') {
            while (!isEmpty(&operators) && peek(&operators)->data != '(') {
                Node* op = pop(&operators);
                op->right = pop(&operands);
                op->left = pop(&operands);
                push(&operands, op);
            }
            pop(&operators); // remove '('
        }
    }
}
```

```
while (!isEmpty(&operators)) {
    Node* op = pop(&operators);
    op->right = pop(&operands);
    op->left = pop(&operands);
    push(&operands, op);
}
return pop(&operands); // root of expression tree
}

// Preorder traversal for prefix
void preorder(Node* root) {
    if (root != NULL) {
        printf("%c", root->data);
        preorder(root->left);
        preorder(root->right);
    }
}

// Postorder traversal for postfix
void postorder(Node* root) {
    if (root != NULL) {
        postorder(root->left);
        postorder(root->right);
        printf("%c", root->data);
    }
}

// Main function
int main() {
    char exp[100];
    printf("Enter a fully parenthesized arithmetic expression: ");
    fgets(exp, 100, stdin);
    exp[strcspn(exp, "\n")] = 0; // remove newline
    Node* root = infixToExprTree(exp);
    printf("Prefix expression: ");
    preorder(root);
    printf("\n");
}
```

```
printf("Postfix expression: ");  
postorder(root);  
printf("\n");  
return 0;  
}
```

OUTPUT

Example1:

(a+(b*c))

Prefix expression: +a*bc

Postfix expression: abc*+

Example2:

((a+b)*(c-d))

Prefix expression: *+ab-cd

Postfix expression: ab+cd-*

MODEL VIVA QUESTIONS

1. What is a binary tree?
2. What are the components of a binary tree node?
3. What is an expression tree?
4. What is the difference between a binary tree and a binary search tree?
5. What is the maximum number of children in a binary tree?
6. What is the root node?

RESULT

The binary expression tree is successfully created for the given arithmetic expression. By traversing the binary tree, the equivalent prefix (preorder traversal) and postfix (postorder traversal) expressions are generated correctly.

EXP NO:8**DICTIONARY USING BINARY SEARCH TREE**

AIM: Implement a dictionary of word-meaning pairs using binary search trees

LEARNING OUTCOME

Students will be able to implement a dictionary of word–meaning pairs using a Binary Search Tree (BST) and perform efficient insertion, searching, deletion, and traversal operations.

ALGORITHM

1. Start
2. Initialize
 - Set root $\leftarrow$ NULL.
3. Repeat the following until user chooses Exit:
 - 3.1 Display menu
 1. Insert Word
 2. Search Word
 3. Display Dictionary
 4. Exit
 - 3.2 Read user's choice
4. If choice = 1 (Insert Word)
 1. Read word.
 2. Read meaning.
 3. Call insert(root, word, meaning) and update root.

Insert Operation

1. If root = NULL, create a new node and return it.
2. Compare word with root->word:
 - If word < root->word, insert into left subtree.
 - If word > root->word, insert into right subtree.
 - If equal, display “Word already exists.”
3. Return updated root.

5. If choice = 2 (Search Word)

1. Read word to search.
2. Call search(root, word).

Search Operation

1. If current node is NULL, return NULL.
2. Compare word with current node's word:
 - If equal → return node.
 - If smaller → search left subtree.
 - If larger → search right subtree.
3. If result is found, display meaning; otherwise display “Word not found.”

6. If choice = 3 (Display Dictionary)

1. Call inorder(root).

Inorder Traversal

1. Traverse left subtree.
2. Print current node's word and meaning.
3. Traverse right subtree.

7. If choice = 4

- Stop program.

8. End

PROGRAM

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

// Node structure
typedef struct Node {
    char word[50];
    char meaning[200];
    struct Node *left, *right;
} Node;

// Function to create a new node
Node* createNode(char *word, char *meaning) {
    Node* newNode = (Node*)malloc(sizeof(Node));
    strcpy(newNode->word, word);
    strcpy(newNode->meaning, meaning);
    newNode->left = newNode->right = NULL;
    return newNode;
}

// Insert node into BST (word alphabetical order)
Node* insert(Node* root, char *word, char *meaning) {
    if (root == NULL)
        return createNode(word, meaning);
    int cmp = strcmp(word, root->word);
    if (cmp < 0)
        root->left = insert(root->left, word, meaning);
    else if (cmp > 0)
        root->right = insert(root->right, word, meaning);
    else
        printf("Word already exists in dictionary!\n");
}
```

```
    return root;
}

// Search for a word in dictionary
Node* search(Node* root, char *word) {
    if (root == NULL)
        return NULL;
    int cmp = strcmp(word, root->word);
    if (cmp == 0)
        return root;
    else if (cmp < 0)
        return search(root->left, word);
    else
        return search(root->right, word);
}

// Display dictionary in alphabetical order
void inorder(Node* root) {
    if (root != NULL) {
        inorder(root->left);
        printf("%s : %s\n", root->word, root->meaning);
        inorder(root->right);
    }
}

int main() {
    Node* root = NULL;
    int choice;
    char word[50], meaning[200];
    while (1) {
        printf("\n--- DICTIONARY MENU (BST) ---\n");
        printf("1. Insert Word\n2. Search Word\n3. Display Dictionary\n4. Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);
        getchar(); // remove newline
        switch (choice) {
            case 1:
```

```
    printf("Enter word: ");
    fgets(word, 50, stdin);
    word[strcspn(word, "\n")] = 0;
    printf("Enter meaning: ");
    fgets(meaning, 200, stdin);
    meaning[strcspn(meaning, "\n")] = 0;
    root = insert(root, word, meaning);
    break;
case 2:
    printf("Enter word to search: ");
    fgets(word, 50, stdin);
    word[strcspn(word, "\n")] = 0;
    Node* result = search(root, word);
    if (result)
        printf("Meaning: %s\n", result->meaning);
    else
        printf("Word not found in dictionary.\n");
    break;
case 3:
    printf("\n--- Dictionary Contents (Alphabetical Order) ---\n");
    inorder(root);
    break;
case 4:
    return 0;
default:
    printf("Invalid choice!\n");
}
}
}
```

OUTPUT

--- DICTIONARY MENU (BST) ---

1. Insert Word
2. Search Word

3. Display Dictionary

4. Exit

Enter choice: 1

Enter word: apple

Enter meaning: A fruit

Enter choice: 1

Enter word: cat

Enter meaning: A small animal

Enter choice: 3

--- Dictionary Contents (Alphabetical Order) ---

apple : A fruit

cat : A small animal

Enter choice: 2

Enter word: apple

Meaning: A fruit

MODEL VIVA QUESTIONS

1. What is a Binary Search Tree (BST)?
2. How is a BST different from a binary tree?
3. What are the disadvantages of a BST?
4. What are the applications of BST?

RESULT

The dictionary of word–meaning pairs is successfully implemented using a Binary Search Tree (BST). The insertion, search, deletion, and display operations are performed efficiently by maintaining the BST property, allowing quick retrieval of word–meaning pairs.

EXP NO:9**SHORTEST DISTANCE TO LANDMINE IN A MAZE(MULTI-SOURCE BFS)**

AIM: To Find the shortest distance of every cell from a landmine inside a maze.

LEARNING OUTCOME

By the end of this experiment, students will be able to determine the shortest distance of every cell from a landmine in a maze using graph traversal algorithms and apply efficient pathfinding techniques to solve grid-based problems.

ALGORITHM

1. Start
2. Input
 - Read the number of rows R and columns C.
 - Read the maze grid grid[R][C]
(0 = safe, 1 = landmine, 2 = blocked).
3. Initialize
 - Create a distance matrix dist[R][C] and fill all cells with -1.
 - Create an empty queue Q.
4. Insert All Landmines Into Queue (Multi-Source BFS Setup)

For every cell (i, j):

1. If grid[i][j] == 1 (landmine):
 - Set dist[i][j] = 0
 - Enqueue (i, j)

Blocked cells (2) remain -1.

5. Perform BFS

While queue Q is not empty:

1. Dequeue a cell (x, y)
2. For each of the 4 possible movements:

- $(x+1, y) \rightarrow$ down

- $(x-1, y) \rightarrow$ up

- $(x, y+1) \rightarrow$ right

- $(x, y-1) \rightarrow$ left

3. Let the new cell be (nx, ny)

Check:

- (nx, ny) is inside the grid

- $\text{grid}[nx][ny] == 0$ (safe cell)

- $\text{dist}[nx][ny] == -1$ (not visited yet)

4. If valid:

- Set $\text{dist}[nx][ny] = \text{dist}[x][y] + 1$

- Enqueue (nx, ny)

6. After BFS Ends

- $\text{dist}[][]$ now stores the shortest distance of each cell from the nearest landmine.
- Cells that remain -1 are either blocked or unreachable.

7. Output

Print the $\text{dist}[][]$ matrix.

8. End

PROGRAM

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#define MAX 1000
```

```
#define INF -1
```

```
// Directions (up, down, left, right)
```

```
int dx[4] = {-1, 1, 0, 0};
```

```
int dy[4] = {0, 0, -1, 1};
```

```
typedef struct {
```

```
int x, y;
} QueueNode;
QueueNode queue[MAX * MAX];
int front = 0, rear = 0;
void enqueue(int x, int y) {
    queue[rear].x = x;
    queue[rear].y = y;
    rear++;
}
QueueNode dequeue() {
    return queue[front++];
}
int isEmpty() {
    return front == rear;
}
int main() {
    int R, C;
    printf("Enter rows and columns of maze: ");
    scanf("%d %d", &R, &C);
    int grid[R][C];
    int dist[R][C];
    printf("Enter grid values (0 = safe, 1 = landmine, 2 = blocked):\n");
    for (int i = 0; i < R; i++) {
        for (int j = 0; j < C; j++) {
            scanf("%d", &grid[i][j]);
            dist[i][j] = INF; // initialize distance array
        }
    }
    // Step 1: Push all landmines into queue
    for (int i = 0; i < R; i++) {
        for (int j = 0; j < C; j++) {
            if (grid[i][j] == 1) {
                dist[i][j] = 0; // distance to itself is 0
                enqueue(i, j); // multi-source BFS
            }
        }
    }
}
```

```
    }
}
}
// Step 2: Perform BFS
while (!isEmpty()) {
    QueueNode node = dequeue();
    int x = node.x;
    int y = node.y;
    for (int k = 0; k < 4; k++) {
        int nx = x + dx[k];
        int ny = y + dy[k];
        // Check boundaries
        if (nx >= 0 && nx < R && ny >= 0 && ny < C) {
            // Visit only safe cells (0) and unvisited
            if (grid[nx][ny] == 0 && dist[nx][ny] == INF) {
                dist[nx][ny] = dist[x][y] + 1;
                enqueue(nx, ny);
            }
        }
    }
}

// Step 3: Print the distance matrix
printf("\nShortest distance of each cell from landmine:\n");
for (int i = 0; i < R; i++) {
    for (int j = 0; j < C; j++) {
        printf("%3d ", dist[i][j]);
    }
    printf("\n");
}

return 0;
}
```

OUTPUT

Enter rows and columns of maze: 4 4

Enter grid values (0 = safe, 1 = landmine, 2 = blocked):

```
1 0 0 2
0 0 2 0
0 2 0 0
1 0 0 0
```

Shortest distance of each cell from landmine:

```
0 1 2 -1
1 2 -1 2
1 -1 1 1
0 1 2 2
```

MODEL VIVA QUESTIONS

1. What is a graph?
2. How can a maze be represented as a graph?
3. What is Breadth-First Search (BFS)?
4. Why is BFS preferred for finding the shortest path in an unweighted graph?
5. What is the difference between BFS and DFS?

RESULT

The shortest distance from every cell to the nearest landmine inside the maze is successfully computed using an efficient graph traversal algorithm (such as Breadth-First Search). The minimum distance for each reachable cell is calculated correctly while avoiding blocked cells.

EXP NO:10**SOLUTION OF WATER JUG PROBLEM USING BFS**

AIM: We have three containers whose sizes are 10 litres, 7 litres, and 4 litres, respectively. The 7-litre and 4-litre containers start out full of water, but the 10-litre container is initially empty. We are allowed one type of operation: pouring the contents of one container into another, stopping only when the source container is empty or the destination container is full. We want to know if there is a sequence of pourings that leaves exactly 2 litres in the 7 or 4-litre container. Model this as a graph problem and solve.

LEARNING OUTCOME

By the end of this experiment, students will be able to model and solve the water jug problem using graph representation and graph traversal techniques to find a sequence of operations that reaches the desired state.

ALGORITHM

1. Start
2. Define Capacities
 - CAPA = 10 (10 L jug)
 - CAPB = 7 (7 L jug)
 - CAPC = 4 (4 L jug)
3. Initialize
 - Create a visited array `visited[CAPA+1][CAPB+1][CAPC+1]` and initialize all to 0.
 - Create an empty queue Q to perform BFS.
4. Define Initial State
start = (A=0, B=7, C=4)
 - Enqueue start into Q
 - Mark start as visited: `visited[0][7][4] = 1`
5. BFS Loop

While the queue is not empty:

1. Dequeue the front state (a, b, c).
2. Check Goal:
If $b == 2$ or $c == 2$,
→ Goal reached, exit BFS.
3. Generate All Possible Pourings (6 possibilities):
 1. Pour $A \rightarrow B$
 2. Pour $A \rightarrow C$
 3. Pour $B \rightarrow A$
 4. Pour $B \rightarrow C$
 5. Pour $C \rightarrow A$
 6. Pour $C \rightarrow B$
4. For each pouring operation:
 1. Calculate the amount to pour = minimum of source jug content and destination jug free capacity.
 2. Create a new state (newA, newB, newC) after pouring.
 3. If $\text{visited}[\text{newA}][\text{newB}][\text{newC}] == 0$:
 - Enqueue this new state into Q
 - Mark as visited

6. Goal Found

- If BFS reaches a state (a, b, c) with $b == 2$ or $c == 2$:
 1. Stop BFS
 2. Backtrack using the parent information stored in the queue to print the sequence of states from start to goal.

7. No Solution

- If BFS completes and no state meets the goal:
→ Print “No solution found”

8. End

PROGRAM

```
#include <stdio.h>
#include <stdlib.h>
typedef struct {
    int a, b, c; // water levels in 10, 7, 4 litre jugs
} State;
typedef struct Node {
    State state;
    int parent; // index of parent in queue, for backtracking
} Node;
#define CAPA 10
#define CAPB 7
#define CAPC 4
int visited[11][8][5]; // visited[a][b][c]
Node queue[1000];
int front = 0, rear = 0;
// Add state to queue
void enqueue(State s, int parent) {
    queue[rear].state = s;
    queue[rear].parent = parent;
    rear++;
}
// Take from queue
Node dequeue() {
    return queue[front++];
}
// Check if state already visited
int isVisited(State s) {
    return visited[s.a][s.b][s.c];
}
// Mark visited
void markVisited(State s) {
```

```
visited[s.a][s.b][s.c] = 1;
}
// Print a state
void printState(State s) {
    printf("A=%d B=%d C=%d\n", s.a, s.b, s.c);
}
// Pour operation function
State pour(State s, int from, int to) {
    int cap[3] = {CAPA, CAPB, CAPC};
    int amt[3] = {s.a, s.b, s.c};
    int space = cap[to] - amt[to];
    int transfer = amt[from] < space ? amt[from] : space;
    amt[from] -= transfer;
    amt[to] += transfer;
    State ns = {amt[0], amt[1], amt[2]};
    return ns;
}

int isGoal(State s) {
    return (s.b == 2 || s.c == 2);
}

int main() {
    // Initial state
    State start = {0, 7, 4};
    enqueue(start, -1);
    markVisited(start);
    int found = -1;
    while (front < rear) {
        Node curr = dequeue();
        if (isGoal(curr.state)) {
            found = front - 1;
            break;
        }
    }
    // For each pour (6 possible)
```

```
int from[6] = {0,0,1,1,2,2};
int to[6] = {1,2,0,2,0,1};
for (int i = 0; i < 6; i++) {
    State ns = pour(curr.state, from[i], to[i]);
    if (!isVisited(ns)) {
        markVisited(ns);
        enqueue(ns, front - 1);
    }
}

if (found == -1) {
    printf("No solution found.\n");
    return 0;
}

// Backtrack to print solution path
printf("\nSolution Found!\nSequence of States:\n");
int path[500], idx = 0;
int p = found;
while (p != -1) {
    path[idx++] = p;
    p = queue[p].parent;
}

// Print path in correct order
for (int i = idx - 1; i >= 0; i--) {
    printState(queue[path[i]].state);
}

return 0;
}
```

OUTPUT

Solution Found!

Sequence of States:

A=0 B=7 C=4

A=4 B=7 C=0

A=4 B=3 C=4

A=8 B=3 C=0

A=8 B=0 C=3

A=1 B=7 C=3

A=1 B=6 C=4

A=5 B=6 C=0

A=5 B=2 C=4 <-- GOAL (2 L in 7-L jug)

MODEL VIVA QUESTIONS

1. What is a graph?
2. How is the water jug problem modeled as a graph?
3. What does each node (state) represent?
4. What does an edge represent in the graph?
5. Why is graph traversal used to solve this problem?

RESULT

The water jug problem is successfully modeled as a state-space graph, where each state represents the amount of water in the 10-litre, 7-litre, and 4-litre containers. By applying valid pouring operations and traversing the graph using Breadth-First Search (BFS) or Depth-First Search (DFS), a sequence of moves is found that leaves exactly 2 litres in either the 7-litre or 4-litre container.

EXP NO:11**FIND AND REPLACE IN TEXT**

AIM: Implement the find and replace feature in a text editor.

LEARNING OUTCOME

By the end of this experiment, students will be able to implement the find and replace feature in a text editor using appropriate string processing techniques and efficiently manipulate text data.

ALGORITHM

1. Start

2. Input

1. Read the text from the user and store it in a string text.
2. Read the word to find and store it in findWord.
3. Read the replacement word and store it in replaceWord.

3. Initialize

1. Create an empty string result to store the modified text.
2. Set a pointer pos pointing to the start of text.

4. Loop: Search and Replace

Repeat until all occurrences of findWord are processed:

1. Use strstr(pos, findWord) to find the next occurrence of findWord in text.
 - Let found point to the start of the found word.
 - If no occurrence is found, exit the loop.
2. Copy all characters from pos up to found into result.
3. Append replaceWord to result.
4. Move pos to the character after the found word.

5. Append Remaining Text

- Copy any remaining text after the last occurrence into result.

6. Copy Result Back

- Copy result back into text to update the original text.

7. Output

- Print the modified text.

8. End

PROGRAM

```
#include <stdio.h>
#include <string.h>
#define MAX_TEXT 1000
#define MAX_WORD 100
// Function to replace all occurrences of 'findWord' with 'replaceWord' in text
void findAndReplace(char *text, const char *findWord, const char *replaceWord) {
    char result[MAX_TEXT] = ""; // Store the modified text
    char *pos = text;
    char *found;
    int findLen = strlen(findWord);
    int replaceLen = strlen(replaceWord);
    while ((found = strstr(pos, findWord)) != NULL) {
        // Copy text before the found word
        strncat(result, pos, found - pos);
        // Append the replacement word
        strcat(result, replaceWord);
        // Move the pointer after the found word
        pos = found + findLen;
    }
    // Append the remaining text
    strcat(result, pos);
    // Copy result back to original text
    strcpy(text, result);
}
```

```
int main() {  
    char text[MAX_TEXT];  
    char findWord[MAX_WORD], replaceWord[MAX_WORD];  
    printf("Enter the text:\n");  
    fgets(text, MAX_TEXT, stdin);  
    text[strcspn(text, "\n")] = 0; // Remove newline  
    printf("Enter the word to find: ");  
    fgets(findWord, MAX_WORD, stdin);  
    findWord[strcspn(findWord, "\n")] = 0;  
    printf("Enter the replacement word: ");  
    fgets(replaceWord, MAX_WORD, stdin);  
    replaceWord[strcspn(replaceWord, "\n")] = 0;  
    findAndReplace(text, findWord, replaceWord);  
    printf("\nModified Text:\n%s\n", text);  
    return 0;  
}
```

OUTPUT

Enter the text:

I love programming. Programming is fun.

Enter the word to find: programming

Enter the replacement word: coding

Modified Text:

I love coding. Programming is fun.

MODEL VIVA QUESTIONS

1. What is a string?
2. How are strings stored in C?
3. What is the difference between a character and a string?
4. What is a null character (`'\0'`)?
5. What are the common string handling functions in C?
6. What is the purpose of `strlen()`?

7. What is the purpose of `strcmp()`?

RESULT

The Find and Replace feature is successfully implemented. The program searches for the specified word or string in the given text and replaces all its occurrences with the new word or string, producing the updated text correctly.

EXP NO:12**BINARY SEARCH**

AIM: Given an array of sorted items, implement an efficient algorithm to search for specific item in the array.

LEARNING OUTCOME

By the end of this experiment, students will be able to implement an efficient search algorithm on a sorted array and analyze its performance in locating a specific element.

ALGORITHM

1. Start
2. Initialize Pointers low = 0 and high = n - 1
3. Repeat While low $\leq$ high
 - a. Calculate middle index , mid = low + (high - low) / 2
 - b. Compare arr[mid] with x:
 - i. If arr[mid] == x $\rightarrow$ return mid (item found)
 - ii. Else if arr[mid] < x $\rightarrow$ search right half, ie low = mid + 1
 - iii. Else if arr[mid] > x $\rightarrow$ search left half, ie high = mid - 1
4. If loop ends
 - a. If low > high $\rightarrow$ return -1 (item not found)
5. end

PROGRAM

```
#include <stdio.h>

int binarySearch(int arr[], int n, int key) {
    int low = 0, high = n - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (arr[mid] == key)
            return mid; // item found
        else if (arr[mid] < key)
            low = mid + 1; // search right half
        else
            high = mid - 1; // search left half
    }
}
```

```
    return -1; // item not found
}

int main() {
    int n, key;
    printf("Enter size of array: ");
    scanf("%d", &n);

    int arr[n];
    printf("Enter %d elements in sorted order:\n", n);
    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i]);
    printf("Enter item to search: ");
    scanf("%d", &key);
    int index = binarySearch(arr, n, key);
    if (index != -1)
        printf("Item %d found at index %d\n", key, index);
    else
        printf("Item %d not found in array\n", key);
    return 0;
}
```

OUTPUT

Enter size of array: 6

Enter 6 elements in sorted order:

1 3 5 7 9 11

Enter item to search: 7

Item 7 found at index 3

MODEL VIVA QUESTIONS

1. What is binary search?
2. What is the prerequisite for binary search?
3. Why must the array be sorted before applying binary search?
4. How does binary search work?

5. What is the difference between binary search and linear search?

RESULT

The binary search algorithm is successfully implemented. The given element is searched by repeatedly dividing the sorted array into two halves until the element is found or the search interval becomes empty

EXP NO:13**SORTING ALGORITHMS WITH STEP COUNT**

AIM: Implement Bubble sort, Insertion Sort, Radix sort, Quick Sort, and Merge Sort and compare the number of steps involved.

LEARNING OUTCOME

By the end of this experiment, students will be able to implement Bubble Sort, Insertion Sort, Radix Sort, Quick Sort, and Merge Sort, compare the number of steps involved, and evaluate their efficiency based on algorithmic performance.

ALGORITHM

1. Start
2. Input
 - Read the size of the array n.
 - Read n elements into array arr
3. For Each Sorting Algorithm

Do the following steps for each sort:

3.1Bubble Sort

Initialize step counter steps = 0.

For i = 0 to n-2:

- For j = 0 to n-i-2:
 - i. Increment steps (comparison).
 - ii. If arr[j] > arr[j+1]:
 1. Swap arr[j] and arr[j+1].
 2. Increment steps (swap).

Return steps for Bubble Sort.

3.2Insertion Sort

Initialize step counter steps = 0.

For i = 1 to n-1:

- Store key = arr[i].
- Set j = i - 1.
- While j >= 0 and arr[j] > key:
 - i. Increment steps (comparison).
 - ii. Move arr[j] to arr[j+1].
 - iii. Increment steps (shift).
 - iv. Decrement j.
- Insert key at arr[j+1].
- Increment steps (insertion).

Return steps for Insertion Sort.

3.3 Radix Sort

Initialize step counter steps = 0.

Find the maximum element m in the array.

For each digit exp from least significant to most:

Initialize count[10] = {0}.

Count occurrences of digits at current place value; increment steps for each count.

Compute cumulative counts.

Place elements into output array according to count; increment steps for each placement.

Copy output back to original array.

Return steps for Radix Sort.

3.4 Quick Sort

Initialize step counter quickSteps = 0.

Function quickSort(arr, low, high):

- If low < high:
 - Partition the array around pivot:
 - Increment quickSteps for each comparison.
 - Increment quickSteps for each swap.

- Recursively quickSort left and right subarrays.

Return quickSteps.

3.5 Merge Sort

Initialize step counter mergeSteps = 0.

Function mergeSort(arr, l, r):

If $l < r$:

- Find middle $m = (l+r)/2$.
- Recursively call mergeSort(arr, l, m) and mergeSort(arr, m+1, r).
- Merge the two halves:
 - Increment mergeSteps for each comparison.
 - Increment mergeSteps for each placement into the merged array.

Return mergeSteps.

Print the number of steps for each sorting algorithm.

End

PROGRAM

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
// ----- BUBBLE SORT -----
int bubbleSort(int arr[], int n) {
    int steps = 0;
    for(int i = 0; i < n-1; i++) {
        for(int j = 0; j < n-i-1; j++) {
            steps++; // comparison
            if(arr[j] > arr[j+1]) {
                int temp = arr[j];
                arr[j] = arr[j+1];
                arr[j+1] = temp;
                steps++; // swap counted
            }
        }
    }
}
```

```
}
return steps;
}

// ----- INSERTION SORT -----
int insertionSort(int arr[], int n) {
    int steps = 0;
    for(int i = 1; i < n; i++) {
        int key = arr[i];
        int j = i - 1;
        while(j >= 0 && arr[j] > key) {
            steps++; // comparison
            arr[j+1] = arr[j];
            steps++; // shift counted
            j--;
        }
        if(j >= 0) steps++; // last comparison failed
        arr[j+1] = key;
        steps++; // insertion counted
    }
    return steps;
}

// ----- RADIX SORT -----
int getMax(int arr[], int n) {
    int mx = arr[0];
    for(int i=1;i<n;i++)
        if(arr[i]>mx) mx = arr[i];
    return mx;
}

int radixSort(int arr[], int n) {
    int steps = 0;
    int m = getMax(arr,n);
    for(int exp=1; m/exp>0; exp*=10) {
        int output[n];
        int i, count[10] = {0};
```

```
for(i=0;i<n;i++) {
    count[(arr[i]/exp)%10]++;
    steps++; // count occurrence
}
for(i=1;i<10;i++) count[i] += count[i-1];

for(i=n-1;i>=0;i--) {
    output[count[(arr[i]/exp)%10]-1] = arr[i];
    count[(arr[i]/exp)%10]--;
    steps++; // placement counted
}
for(i=0;i<n;i++) arr[i] = output[i];
}
return steps;
}
// ----- QUICK SORT -----
int quickSteps = 0;
int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = low-1;
    for(int j=low;j<high;j++) {
        quickSteps++; // comparison
        if(arr[j]<pivot) {
            i++;
            int temp = arr[i];
            arr[i]=arr[j];
            arr[j]=temp;
            quickSteps++; // swap counted
        }
    }
    int temp = arr[i+1];
    arr[i+1]=arr[high];
    arr[high]=temp;
```

```
    quickSteps++; // swap counted
    return i+1;
}

void quickSort(int arr[], int low, int high) {
    if(low<high) {
        int pi = partition(arr,low,high);
        quickSort(arr,low,pi-1);
        quickSort(arr,pi+1,high);
    }
}

// ----- MERGE SORT -----
int mergeSteps = 0;
void merge(int arr[], int l, int m, int r) {
    int n1 = m-l+1;
    int n2 = r-m;
    int L[n1], R[n2];
    for(int i=0;i<n1;i++) L[i]=arr[l+i];
    for(int j=0;j<n2;j++) R[j]=arr[m+1+j];
    int i=0,j=0,k=l;
    while(i<n1 && j<n2) {
        mergeSteps++; // comparison
        if(L[i]<=R[j]) {
            arr[k++] = L[i++];
            mergeSteps++; // placement
        } else {
            arr[k++] = R[j++];
            mergeSteps++; // placement
        }
    }
    while(i<n1) { arr[k++]=L[i++]; mergeSteps++; }
    while(j<n2) { arr[k++]=R[j++]; mergeSteps++; }
}

void mergeSort(int arr[], int l, int r) {
    if(l<r) {
```

```
        int m = l+(r-l)/2;
        mergeSort(arr,l,m);
        mergeSort(arr,m+1,r);
        merge(arr,l,m,r);
    }
}

// ----- COPY ARRAY -----
void copyArray(int src[], int dest[], int n) {
    for(int i=0;i<n;i++) dest[i]=src[i];
}

// ----- PRINT ARRAY -----
void printArray(int arr[], int n) {
    for(int i=0;i<n;i++) printf("%d ",arr[i]);
    printf("\n");
}

// ----- MAIN -----
int main() {
    int n;
    printf("Enter size of array: ");
    scanf("%d",&n);
    int arr[n], temp[n];

    printf("Enter %d elements:\n",n);
    for(int i=0;i<n;i++) scanf("%d",&arr[i]);
    int steps;
    // Bubble Sort
    copyArray(arr,temp,n);
    steps = bubbleSort(temp,n);
    printf("Bubble Sort Steps: %d\n",steps);
    // Insertion Sort
    copyArray(arr,temp,n);
    steps = insertionSort(temp,n);
    printf("Insertion Sort Steps: %d\n",steps);
    // Radix Sort
```

```

copyArray(arr,temp,n);
steps = radixSort(temp,n);
printf("Radix Sort Steps: %d\n",steps);
// Quick Sort
copyArray(arr,temp,n);
quickSteps = 0;
quickSort(temp,0,n-1);
printf("Quick Sort Steps: %d\n",quickSteps);
// Merge Sort
copyArray(arr,temp,n);
mergeSteps = 0;
mergeSort(temp,0,n-1);
printf("Merge Sort Steps: %d\n",mergeSteps);
return 0;
}

```

OUTPUT

Enter size of array: 6

Enter 6 elements:

5 2 9 1 6 3

Bubble Sort Steps: 15

Insertion Sort Steps: 14

Radix Sort Steps: 34

Quick Sort Steps: 12

Merge Sort Steps: 20

Input Array	Bubble Sort	Insertion Sort	Radix Sort	Quick Sort	Merge Sort
5 2 9 1 6 3	15	14	34	12	20
1 2 3 4 5 6	15	11	30	12	20
6 5 4 3 2 1	15	21	34	12	20

3 6 1 9 4 2	15	16	34	12	20
-------------	----	----	----	----	----

10 7 8 9 1 5	15	17	34	12	20
--------------	----	----	----	----	----

MODEL VIVA QUESTIONS

1. What is sorting?
2. What is time complexity?
3. What is space complexity?
4. What is the best sorting algorithm?
5. What are the applications of sorting?

RESULT

The Bubble Sort, Insertion Sort, Radix Sort, Quick Sort, and Merge Sort algorithms are successfully implemented. The given list is sorted correctly using each algorithm, and the number of comparisons, swaps, or passes involved is compared to evaluate their efficiency.

EXP NO:14**PREFERENTIAL CUSTOMER SERVICE(PRIORITY ORDER)**

AIM: The General post office wishes to give preferential treatment to its customers. They have identified the customer categories as Defence personnel, Differently abled, Senior citizen, Ordinary. The customers are to be given preference in the decreasing order - Differently abled, Senior citizen, Defence personnel, Normal person. Generate the possible sequence of completion.

LEARNING OUTCOME

By the end of this experiment, students will be able to implement a priority queue to manage customer service based on predefined priority levels and generate the order of service completion efficiently.

ALGORITHM

1.Start

2.Input

- Read the number of customers n.
- For each customer i (from 1 to n):
 - Read name[i]
 - Read category[i]

3.Assign Priority

- For each customer, assign a numeric priority based on category:
 - Differently Abled → 1 (highest)
 - Senior Citizen → 2
 - Defence Personnel → 3
 - Ordinary → 4 (lowest)

4.Sort Customers

- Sort the customers in ascending order of priority.
- If two or more customers have the same priority, maintain their original arrival order (FIFO).

5.Display Sequence of Completion

- For each customer in the sorted list:
 - Print customer name and category in order.

6.End**PROGRAM**

```
#include <stdio.h>
#include <string.h>
#define MAX 100
// Structure to store customer info
typedef struct {
    char name[50];
    char category[30];
    int priority;
} Customer;
// Function to assign priority based on category
int getPriority(char category[]) {
    if (strcmp(category, "Differently Abled") == 0) return 1;
    else if (strcmp(category, "Senior Citizen") == 0) return 2;
    else if (strcmp(category, "Defence Personnel") == 0) return 3;
    else return 4; // Ordinary
}
// Function to sort customers by priority (ascending)
void sortByPriority(Customer customers[], int n) {
    Customer temp;
    for(int i=0;i<n-1;i++) {
        for(int j=i+1;j<n;j++) {
            if(customers[i].priority > customers[j].priority) {
                temp = customers[i];
                customers[i] = customers[j];
                customers[j] = temp;
            }
        }
    }
}
```

```
}

int main() {
    int n;
    Customer customers[MAX];
    printf("Enter number of customers: ");
    scanf("%d", &n);
    getchar(); // to remove newline
    // Input customer details
    for(int i=0;i<n;i++) {
        printf("Enter name of customer %d: ", i+1);
        fgets(customers[i].name, 50, stdin);
        customers[i].name[strcspn(customers[i].name, "\n")] = 0; // remove newline
        printf("Enter category (Differently Abled / Senior Citizen / Defence Personnel / Ordinary): ");
        fgets(customers[i].category, 30, stdin);
        customers[i].category[strcspn(customers[i].category, "\n")] = 0;
        customers[i].priority = getPriority(customers[i].category);
    }
    // Sort customers by priority
    sortByPriority(customers, n);
    // Display sequence of completion
    printf("\nSequence of completion:\n");
    for(int i=0;i<n;i++) {
        printf("%d. %s (%s)\n", i+1, customers[i].name, customers[i].category);
    }
    return 0;
}
```

OUTPUT

Enter number of customers: 6

Enter name of customer 1: Alice

Enter category (Differently Abled / Senior Citizen / Defence Personnel / Ordinary): Ordinary

Enter name of customer 2: Bob

Enter category (Differently Abled / Senior Citizen / Defence Personnel / Ordinary): Senior Citizen

Enter name of customer 3: Charlie

Enter category (Differently Abled / Senior Citizen / Defence Personnel / Ordinary): Differently Abled

Enter name of customer 4: David

Enter category (Differently Abled / Senior Citizen / Defence Personnel / Ordinary): Defence Personnel

Enter name of customer 5: Eve

Enter category (Differently Abled / Senior Citizen / Defence Personnel / Ordinary): Senior Citizen

Enter name of customer 6: Frank

Enter category (Differently Abled / Senior Citizen / Defence Personnel / Ordinary): Ordinary

Sequence of completion:

1. Charlie (Differently Abled)
2. Bob (Senior Citizen)
3. Eve (Senior Citizen)
4. David (Defence Personnel)
5. Alice (Ordinary)
6. Frank (Ordinary)

(Each customer is assigned a **priority** based on category.

Customers are sorted by **priority ascending**, so highest priority (Differently Abled) comes first.

Within the same category, **original arrival order is maintained**.

The program prints the **sequence of completion** according to preferential treatment)

MODEL VIVA QUESTIONS

1. What is a priority queue?
2. How is a priority queue different from a normal queue?
3. What is the principle of a priority queue?
4. What are the applications of a priority queue?
5. Which data structures can be used to implement a priority queue?
6. What is the difference between a queue and a priority queue?

RESULT

The program generates the sequence of customer service based on priority, ensuring that higher-priority customers are served before lower-priority customers.

EXP NO:15**SPELL CHECKER USING HASH TABLE**

AIM: Implement a spell checker using a hash table to store a dictionary of words for fast lookup. Implement functions to check if a word is valid and to suggest corrections for misspelled words..

LEARNING OUTCOME

By the end of this experiment, students will be able to implement a spell checker using a hash table for efficient word lookup, validate words against a dictionary, and generate suitable suggestions for misspelled words

ALGORITHM

1. Start
2. Initialize
 - Create a hash table of size **TABLE_SIZE**.
 - Each entry contains a linked list to handle collisions (chaining).
3. Load Dictionary
 1. Open the dictionary file.
 2. For each word in the file:
 - Compute hash index using a hash function.
 - Insert the word into the linked list at that index.
 3. Close the file.
4. Input Word
 1. Prompt the user to enter a word to check.
 2. Repeat until the user enters **"exit"**.
5. Check Word
 1. Compute the hash index for the input word.

2. Search the linked list at that index:

- If word exists → Print "Word is spelled correctly."
- Else → Print "Word NOT found in dictionary." and go to Step 6.

6. Suggest Corrections

1. Initialize an empty list of suggestions.

2. Generate possible words by:

- Replacing each character in the word with a-z.
- Deleting each character in the word.
- Inserting a character a-z at each position.

3. For each generated word:

- If the word exists in the dictionary → add it to suggestions.

4. Display all suggestions (or "No suggestions found" if none).

7. Repeat

- Go back to Step 4 to check another word.

8. End

PROGRAM

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#define TABLE_SIZE 1000
#define WORD_LEN 50
// Hash table node
typedef struct Node {
    char word[WORD_LEN];
    struct Node* next;
} Node;
Node* hashTable[TABLE_SIZE];
```

```
// ----- Hash Function -----
unsigned int hash(char* str) {
    unsigned int hash = 0;
    while (*str) {
        hash = (hash * 31 + tolower(*str)) % TABLE_SIZE;
        str++;
    }
    return hash;
}

// ----- Insert word into hash table -----
void insert(char* word) {
    unsigned int index = hash(word);
    Node* newNode = (Node*)malloc(sizeof(Node));
    strcpy(newNode->word, word);
    newNode->next = hashTable[index];
    hashTable[index] = newNode;
}

// ----- Search word in hash table -----
int search(char* word) {
    unsigned int index = hash(word);
    Node* temp = hashTable[index];
    while(temp) {
        if(strcmp(temp->word, word) == 0)
            return 1; // word found
        temp = temp->next;
    }
    return 0; // word not found
}

// ----- Load dictionary -----
void loadDictionary(char* filename) {
    FILE* file = fopen(filename, "r");
    if(!file) {
        printf("Error opening dictionary file.\n");
        return;
    }
}
```

```
}
char word[WORD_LEN];
while(fscanf(file, "%s", word) != EOF) {
    insert(word);
}
fclose(file);
}
// ----- Suggest words (simple edits) -----
void suggest(char* word) {
    int found = 0;
    char temp[WORD_LEN];
    printf("Did you mean:\n");
    // Try replacing each character with a-z
    for(int i=0; word[i]!='\0'; i++) {
        char original = word[i];
        for(char c='a'; c<='z'; c++) {
            if(c == original) continue;
            strcpy(temp, word);
            temp[i] = c;
            if(search(temp)) {
                printf(" %s\n", temp);
                found = 1;
            }
        }
    }
    // Try deleting each character
    for(int i=0; word[i]!='\0'; i++) {
        int k=0;
        for(int j=0; word[j]!='\0'; j++) {
            if(j!=i) temp[k++] = word[j];
        }
        temp[k] = '\0';
        if(search(temp)) {
            printf(" %s\n", temp);
        }
    }
}
```

```
        found = 1;
    }
}
// Try inserting a-z at each position
int len = strlen(word);
for(int i=0; i<=len; i++) {
    for(char c='a'; c<='z'; c++) {
        int k=0;
        for(int j=0; j<i; j++) temp[k++] = word[j];
        temp[k++] = c;
        for(int j=i; j<len; j++) temp[k++] = word[j];
        temp[k] = '\0';
        if(search(temp)) {
            printf(" %s\n", temp);
            found = 1;
        }
    }
}
if(!found) printf(" (No suggestions found)\n");
}
// ----- Main -----
int main() {
    char word[WORD_LEN];
    loadDictionary("dictionary.txt"); // dictionary file with one word per line
    while(1) {
        printf("\nEnter word to check (or type 'exit' to quit): ");
        scanf("%s", word);
        if(strcmp(word, "exit") == 0) break;
        if(search(word)) {
            printf("%s' is spelled correctly.\n", word);
        } else {
            printf("%s' is NOT in the dictionary.\n", word);
            suggest(word);
        }
    }
}
```

```
}  
    return 0;  
}
```

OUTPUT

This assumes `dictionary.txt` contains these sample words:

apple
banana
cat
dog
hello
world
sample
simple

Enter word to check (or type 'exit' to quit): apple
'apple' is spelled correctly.

Enter word to check (or type 'exit' to quit): aplpe
'aplpe' is NOT in the dictionary.

Did you mean:

apple

Enter word to check (or type 'exit' to quit): bannana
'bannana' is NOT in the dictionary.

Did you mean:

banana

Enter word to check (or type 'exit' to quit): sampel
'samel' is NOT in the dictionary.

Did you mean:

sample

simple

Enter word to check (or type 'exit' to quit): dog

'dog' is spelled correctly.

Enter word to check (or type 'exit' to quit): exit

MODEL VIVA QUESTIONS

1. What is a hash table?
2. What is a hash function?
3. Why is a hash table used for a spell checker?
4. What are the advantages of a hash table?
5. What are the disadvantages of a hash table?
6. What is a hash key?

RESULT

The spell checker is successfully implemented using a hash table to store the dictionary of words. The program efficiently checks whether a given word is present in the dictionary and reports whether it is valid. If a word is misspelled, the program suggests suitable corrections based on the available dictionary entries.

EXP NO:16**MEMORY ALLOCATOR AND GARBAGE COLLECTOR**

AIM: Simulation of a basic memory allocator and garbage collector using doubly linked list

LEARNING OUTCOME

By the end of this experiment, students will be able to simulate a basic memory allocator and garbage collector using a doubly linked list and understand dynamic memory management through allocation, deallocation, and memory reuse.

ALGORITHM

1. Initialize Memory

CREATE_MEMORY(totalSize)

1. Create a new block B.
2. Set B.size = totalSize.
3. Set B.free = 1 (mark block as free).
4. Set B.prev = NULL and B.next = NULL.
5. Set head = B.

2. Allocate Memory

ALLOCATE(size)

1. Set curr = head.
2. Repeat until curr becomes NULL:
 - If curr.free == 1 and curr.size >= size:
 - If curr.size == size:
 1. Set curr.free = 0.
 2. Return "Allocation successful".
 - Else (curr.size > size):

1. Create a new block NEW.
2. Set NEW.size = curr.size - size.
3. Set NEW.free = 1.
4. Insert NEW after curr.
5. Set curr.size = size.
6. Set curr.free = 0.
7. Return "Allocation successful".

- Otherwise, move to next block: curr = curr.next.

3. If loop ends, return "Allocation failed: insufficient memory".

3. Free Memory

FREE(size)

1. Set curr = head.
2. Traverse the list:
 - If curr.free == 0 and curr.size == size:
 1. Set curr.free = 1.
 2. Return "Block freed".
3. If block not found, return "Free failed: block not found".

4. Garbage Collection (Merge Free Blocks)

GARBAGE_COLLECT()

1. Set curr = head.
2. Repeat while curr != NULL and curr.next != NULL:
 - If both curr.free == 1 and curr.next.free == 1:
 - Merge next block into current block:
curr.size = curr.size + curr.next.size
 - Remove curr.next from the list.

- Else:
 - Move to next block: curr = curr.next.

3. End.

5. Display Memory Blocks

DISPLAY()

1. Set curr = head.
2. While curr != NULL:
 - Print:
[Size = curr.size | Status = Free / Allocated]
 - Move to next block: curr = curr.next.
3. End.

PROGRAM

```
#include <stdio.h>
#include <stdlib.h>
typedef struct Block {
    int size;           // size of the block
    int free;           // 1 = free, 0 = allocated
    struct Block *prev;
    struct Block *next;
} Block;
Block *head = NULL;
// ----- Create Memory Pool -----
void createMemory(int totalSize) {
    head = (Block *)malloc(sizeof(Block));
    head->size = totalSize;
    head->free = 1;
    head->prev = head->next = NULL;
}
// ----- Allocate Memory Block -----
void allocate(int size) {
```

```
Block *curr = head;
while (curr != NULL) {
    if (curr->free && curr->size >= size) {
        // If exact match
        if (curr->size == size) {
            curr->free = 0;
            printf("Allocated %d bytes\n", size);
            return;
        }
        // Split block
        Block *newBlock = (Block *)malloc(sizeof(Block));
        newBlock->size = curr->size - size;
        newBlock->free = 1;
        newBlock->next = curr->next;
        newBlock->prev = curr;
        if (curr->next)
            curr->next->prev = newBlock;
        curr->next = newBlock;
        curr->size = size;
        curr->free = 0;
        printf("Allocated %d bytes (split)\n", size);
        return;
    }
    curr = curr->next;
}
printf("Allocation failed! Not enough memory.\n");
}

// ----- Free a block -----
void freeBlock(int size) {
    Block *curr = head;
    while (curr != NULL) {
        if (!curr->free && curr->size == size) {
            curr->free = 1;
            printf("Freed %d bytes\n", size);
        }
    }
}
```

```
        return;
    }
    curr = curr->next;
}
printf("Free failed! Block of size %d not found.\n", size);
}
// ----- Garbage Collector (merge free blocks) -----
void garbageCollector() {
    Block *curr = head;
    while (curr != NULL && curr->next != NULL) {
        if (curr->free && curr->next->free) {
            // merge
            curr->size += curr->next->size;
            Block *temp = curr->next;
            curr->next = temp->next;
            if (temp->next)
                temp->next->prev = curr;
            free(temp);
        } else {
            curr = curr->next;
        }
    }
    printf("Garbage collection completed (free blocks merged).\n");
}
// ----- Display Memory Blocks -----
void display() {
    Block *curr = head;
    printf("\nMemory State:\n");

    while (curr != NULL) {
        printf("[Size: %d | %s] <=> ",
            curr->size,
            curr->free ? "Free" : "Allocated");
        curr = curr->next;
    }
}
```

```
}  
printf("NULL\n");  
}  
  
// ----- Main -----  
int main() {  
    createMemory(100);  
  
    printf("Memory Allocator and Garbage Collector Simulation\n");  
  
    allocate(30);  
    allocate(20);  
    allocate(10);  
    display();  
  
    freeBlock(20);  
    freeBlock(10);  
    display();  
  
    garbageCollector();  
    display();  
  
    allocate(25);  
    display();  
  
    return 0;  
}
```

OUTPUT

Memory Allocator and Garbage Collector Simulation

Allocated 30 bytes

Allocated 20 bytes

Allocated 10 bytes

Memory State:

[Size: 30 | Allocated] <=> [Size: 20 | Allocated] <=> [Size: 10 | Allocated] <=> [Size: 40 | Free] <=> NULL

Freed 20 bytes

Freed 10 bytes

Memory State:

[Size: 30 | Allocated] <=> [Size: 20 | Free] <=> [Size: 10 | Free] <=> [Size: 40 | Free] <=> NULL

Garbage collection completed (free blocks merged).

Memory State:

[Size: 30 | Allocated] <=> [Size: 70 | Free] <=> NULL

Allocated 25 bytes (split)

Memory State:

[Size: 30 | Allocated] <=> [Size: 25 | Allocated] <=> [Size: 45 | Free] <=> NULL

MODEL VIVA QUESTIONS

1. What is memory allocation?
2. What is dynamic memory allocation?
3. What is memory deallocation?
4. Why is memory management important?

RESULT

The basic memory allocator and garbage collector are successfully simulated using a doubly linked list. Memory blocks are allocated and deallocated efficiently, and the garbage collector identifies and reclaims unused memory blocks, making them available for future allocations.

EXP NO:17**FIND MAXIMUM ACTIVITY POINTS FROM SELECTING AT MOST K EVENTS****AIM:**

The CSE dept is organizing a tech fest with so many exciting events. By participating in an event, you can claim for activity points as stipulated by KTU. Each event i gives you $A[i]$ activity points where A is an array. If you are not allowed to participate in more than k events, what's the max number of points that you can earn?

LEARNING OUTCOME

By the end of this experiment, students will be able to design and implement an algorithm to maximize activity points under given constraints and analyze its efficiency using appropriate algorithmic techniques.

ALGORITHM

1. Start
2. Read the number of events n .
3. Read the array $A[0 \dots n-1]$ containing activity points of each event.
4. Read the value of k (maximum number of events allowed).
5. Sort the array A in descending order (so highest points come first).
6. Initialize $\text{sum} = 0$.
7. Repeat for $i = 0$ to $k - 1$:
 Add the value $A[i]$ to sum .
8. Print the value of sum as the maximum activity points.
9. Stop

PROGRAM

```
#include <stdio.h>
#include <stdlib.h>
int compare(const void *a, const void *b) {
```

```
return (*(int*)b - *(int*)a); // descending order
}

int main() {
    int n, k;
    printf("Enter number of events: ");
    scanf("%d", &n);
    int A[n];
    printf("Enter activity points of events:\n");
    for(int i = 0; i < n; i++)
        scanf("%d", &A[i]);
    printf("Enter maximum events you can participate (k): ");
    scanf("%d", &k);
    // Sort array in descending order
    qsort(A, n, sizeof(int), compare);
    int sum = 0;
    for(int i = 0; i < k; i++)
        sum += A[i];
    printf("Maximum activity points you can earn = %d\n", sum);
    return 0;
}
```

OUTPUT

Enter number of events: 6

Enter activity points of events:

10 25 5 40 15 30

Enter maximum events you can participate (k): 3

Maximum activity points you can earn = 95

RESULT

The program successfully determines the maximum activity points that can be earned by selecting at most k events. The events with the highest activity points are chosen, and their points are summed to obtain the maximum possible score.

EXP NO:18**MERGE K SORTED LISTS USING MIN HEAP**

AIM: Merge K sorted lists into a single sorted list using a heap. Use a min-heap to keep track of the smallest element from each list. Repeatedly extract the smallest element and insert the next element from the corresponding list into the heap until all lists are merged.

LEARNING OUTCOME

By the end of this experiment, students will be able to implement the merging of multiple sorted lists using a min-heap, apply heap operations efficiently, and analyze the performance of the algorithm.

ALGORITHM

1. Start
2. Read the number of lists K.
3. For each list i (from 0 to K-1):
 - Read the size of the list n_i
 - Read the sorted elements of the list into array A[i]
4. Create a min-heap of size K.
 - Each heap node stores:
{value, list_index, element_index}
5. Insert the first element of each list into the min-heap.
6. Initialize an empty result list mergedList.
7. While the min-heap is not empty:
 - a. Extract the minimum element from the heap.
 - b. Append the extracted value to mergedList.
 - c. If the extracted element's list has more elements:
 - Insert the next element from the same list into the min-heap.
8. End While
9. Print the merged list mergedList.
10. Stop

PROGRAM

```
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    int value;    // element value
    int list_idx; // which list it belongs to
    int elem_idx; // index in the list
} HeapNode;

typedef struct {
    HeapNode *arr;
    int size;
} MinHeap;

// ----- Swap -----
void swap(HeapNode *a, HeapNode *b) {
    HeapNode temp = *a;
    *a = *b;
    *b = temp;
}

// ----- Heapify -----
void minHeapify(MinHeap *heap, int i) {
    int smallest = i;
    int left = 2*i + 1;
    int right = 2*i + 2;

    if (left < heap->size && heap->arr[left].value < heap->arr[smallest].value)
        smallest = left;
    if (right < heap->size && heap->arr[right].value < heap->arr[smallest].value)
        smallest = right;

    if (smallest != i) {
        swap(&heap->arr[i], &heap->arr[smallest]);
        minHeapify(heap, smallest);
    }
}
```

```
}
// ----- Extract Min -----
HeapNode extractMin(MinHeap *heap) {
    HeapNode root = heap->arr[0];
    heap->arr[0] = heap->arr[heap->size - 1];
    heap->size--;
    minHeapify(heap, 0);
    return root;
}
// ----- Insert Node -----
void insertHeap(MinHeap *heap, HeapNode node) {
    heap->size++;
    int i = heap->size - 1;
    heap->arr[i] = node;
    // Fix min-heap property
    while (i != 0 && heap->arr[(i-1)/2].value > heap->arr[i].value) {
        swap(&heap->arr[i], &heap->arr[(i-1)/2]);
        i = (i-1)/2;
    }
}
// ----- Merge K Sorted Lists -----
void mergeKSortedLists(int **lists, int *sizes, int K) {
    MinHeap heap;
    heap.arr = (HeapNode *)malloc(sizeof(HeapNode) * K);
    heap.size = 0;
    // Insert first element of each list into heap
    for(int i = 0; i < K; i++) {
        if (sizes[i] > 0) {
            HeapNode node = {lists[i][0], i, 0};
            insertHeap(&heap, node);
        }
    }
    printf("Merged List: ");
    while(heap.size > 0) {
```

```
    HeapNode minNode = extractMin(&heap);
    printf("%d ", minNode.value);
    int next_idx = minNode.elem_idx + 1;
    int list_idx = minNode.list_idx;
    if(next_idx < sizes[list_idx]) {
        HeapNode nextNode = {lists[list_idx][next_idx], list_idx, next_idx};
        insertHeap(&heap, nextNode);
    }
}
printf("\n");
free(heap.arr);
}
// ----- Main -----
int main() {
    int K;
    printf("Enter number of lists: ");
    scanf("%d", &K);
    int **lists = (int **)malloc(sizeof(int*) * K);
    int *sizes = (int *)malloc(sizeof(int) * K);
    for(int i = 0; i < K; i++) {
        printf("Enter size of list %d: ", i+1);
        scanf("%d", &sizes[i]);
        lists[i] = (int *)malloc(sizeof(int) * sizes[i]);
        printf("Enter elements of list %d (sorted): ", i+1);
        for(int j = 0; j < sizes[i]; j++)
            scanf("%d", &lists[i][j]);
    }
    mergeKSortedLists(lists, sizes, K);
    for(int i = 0; i < K; i++)
        free(lists[i]);
    free(lists);
    free(sizes);
    return 0;
}
```

OUTPUT

Enter number of lists: 3

Enter size of list 1: 3

Enter elements of list 1 (sorted): 1 4 7

Enter size of list 2: 4

Enter elements of list 2 (sorted): 2 5 8 11

Enter size of list 3: 2

Enter elements of list 3 (sorted): 3 6

Merged List: 1 2 3 4 5 6 7 8 11 11

MODEL VIVA QUESTIONS

1. What is a heap?
2. What is a min-heap?
3. What is a max-heap?
4. What is the difference between a min-heap and a max-heap?
5. What is the heap property?
6. How is a heap represented?
7. What are the applications of heaps?

RESULT

The K sorted lists are successfully merged into a single sorted list using a min-heap. The program repeatedly extracts the smallest element from the heap and inserts the next element from the corresponding list until all elements are merged, producing the final sorted list efficiently.